

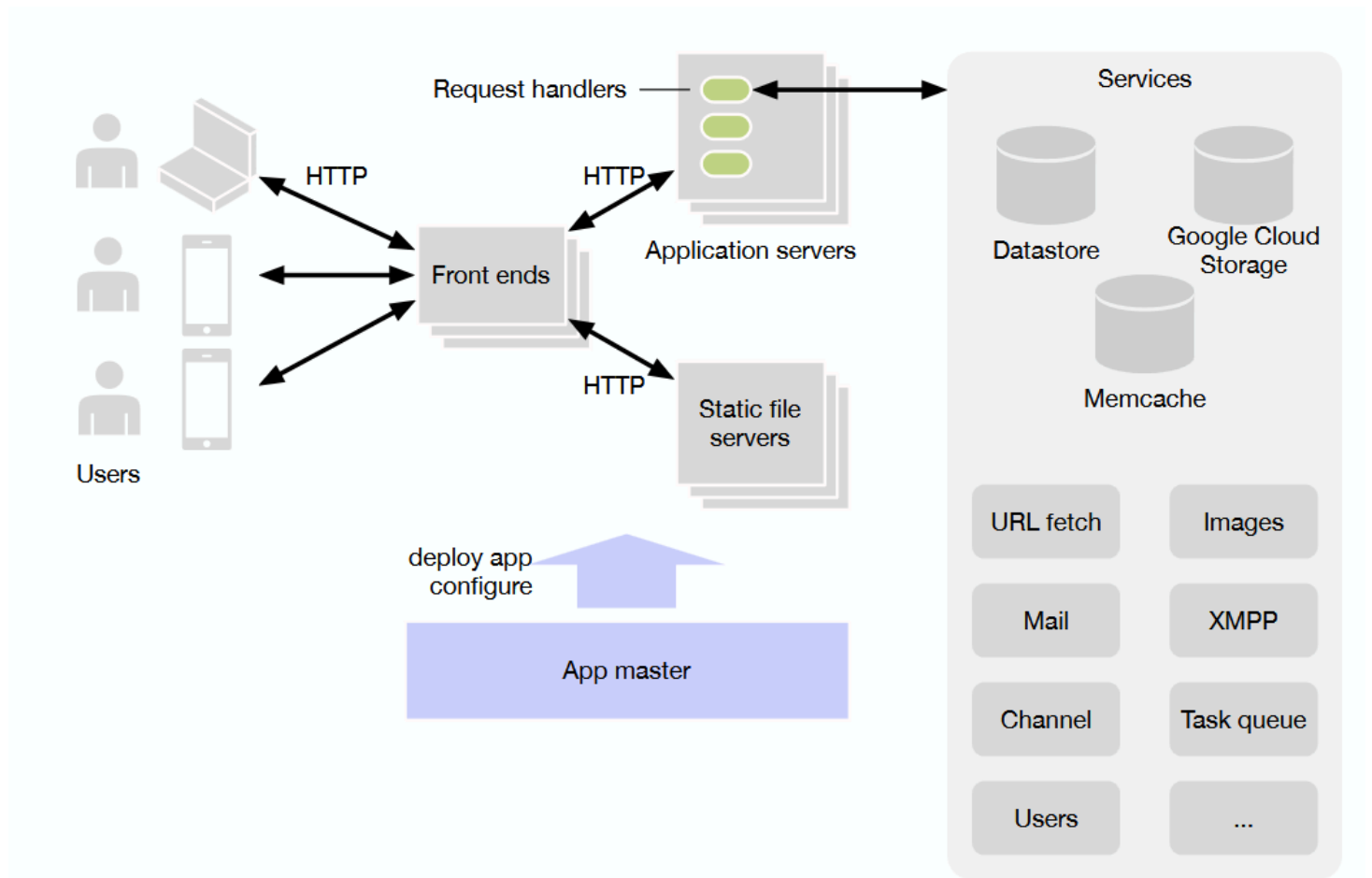
TE 2

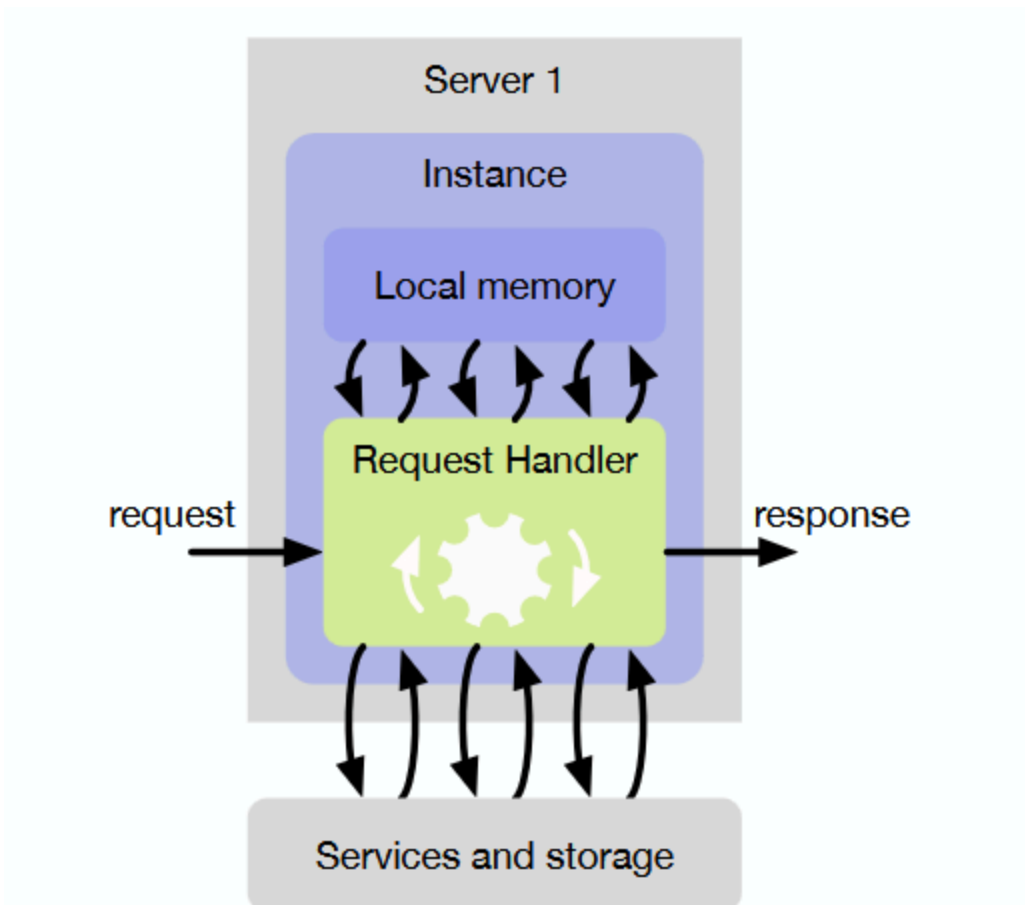
5 Platform-as-a-Service

(PaaS base on the same software are compatible)

- Open source
 - Cloud Foundry
 - OpenShift
- Proprietary
 - Google App engine : for web and mobile back-end
 - autoscaling
 - a system de request handling
 - Request arrive
 - Handler is created
 - Handler create a respond -> transfer to another cloud services
 - Handler is remove
 - It live only in the instance

GOOGLE





scaling to zero

quand y a plus de requet, allou 0 instance

mais du coup quand y a des request qui revienne, ça galère un peut au démarrage (cloud start) (4-6 seconde de delay)

App service

Une app peut ce diviser en service

Data storage

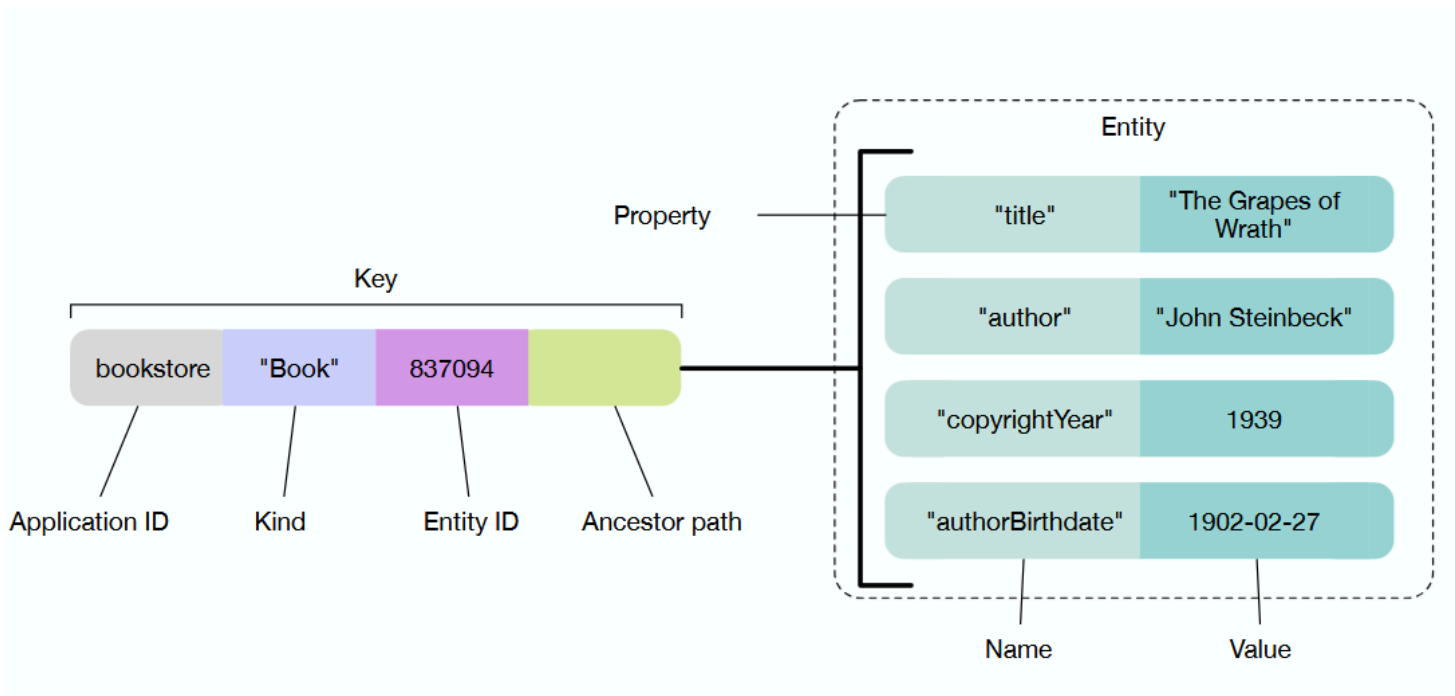
- Google Cloud Storage: Persistent object (= file) storage (similar to AWS S3)
- Google Cloud SQL: Persistent data storage in a single-tenant relational database that is MySQL-compatible (similar to AWS RDS)
- Google Cloud Datastore: Persistent data storage in a multi-tenant NoSQL database

Cloud build

En gro, tu build direct sur le cloud avec google cloud build `gcloud app deploy`

Data store

C'est genre FireBase



- Kind = type

Key composition :

- app identifier
- kind
- entity identifier

Data store API

```
// accessor for the Datastore service
private final Datastore datastore = DatastoreOptions.getDefaultInstance().getService();

// ...

KeyFactory keyFactory = datastore.newKeyFactory().setKind("book");
Key key = datastore.allocateId(keyFactory.newKey());
Date authorBirthdate = new GregorianCalendar(1902, GregorianCalendar.FEBRUARY, 27).getTime();
Entity entity = Entity.newBuilder(key)
    .set("title", "The grapes of wrath")
    .set("author", "John Steinbeck")
    .set("copyrightYear", 1939)
    .set("authorBirthdate", Timestamp.of(authorBirthdate))
    .build();
datastore.put(entity);
```

Class declaration

```
@Entity(name = "Book")
public class Book {
    @Id
    private String isbn;
    private String title;
    private String author;
    private int copyrightYear;
    private Date authorBirthdate;
    // ... constructors, accessors
}
```

Store class declaration

```
EntityManager em = null;
try {
    em = emf.createEntityManager();
    Book book = new Book();
    book.setTitle("The Grapes of Wrath");
    // ...
    em.persist(book);
} finally {
    if (em != null)
        em.close();
}
```

6 1/2 Database-as-a-Service and NoSQL

Scaling a database

we need to scale :

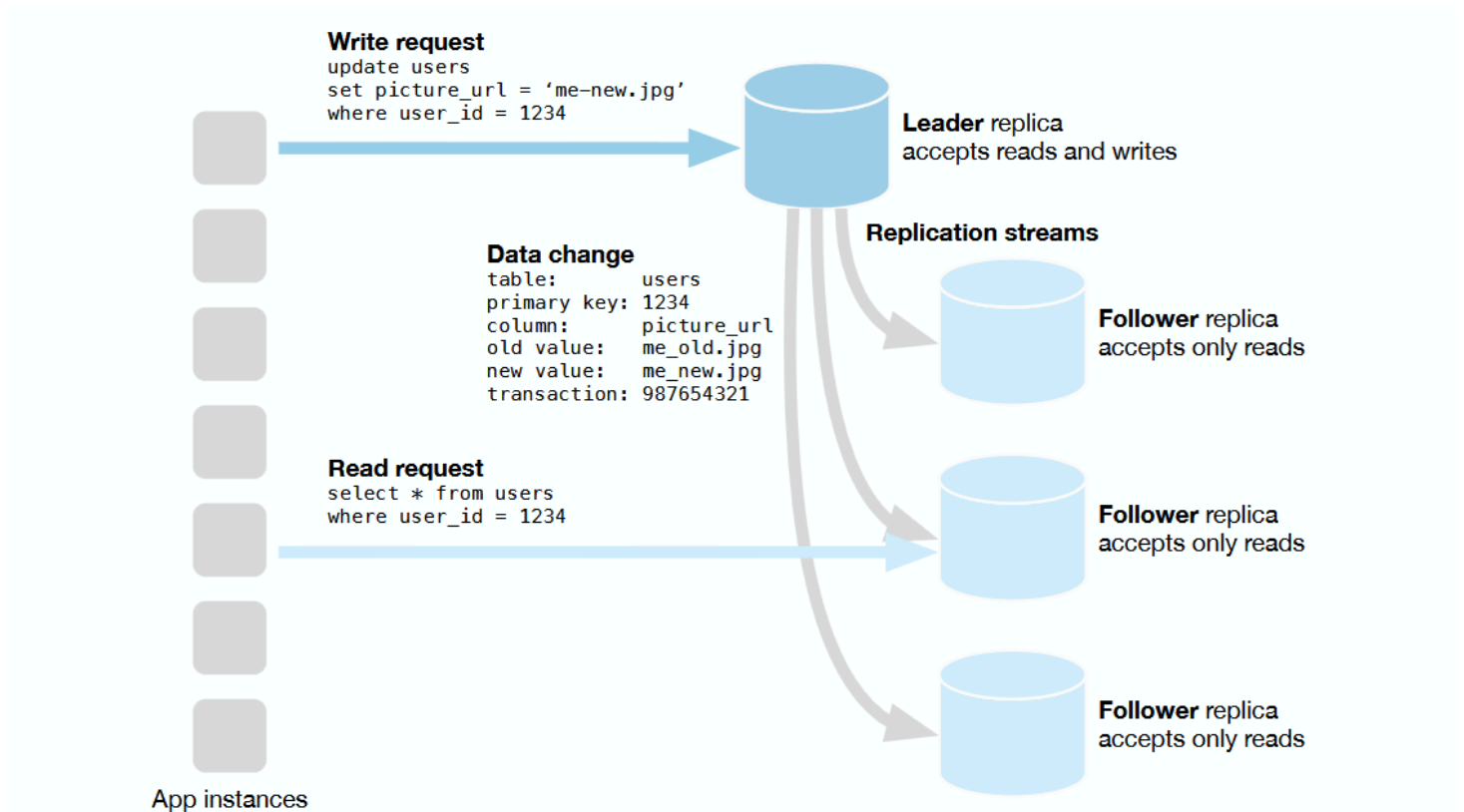
- Storage capacity
- Read request
- Write request

Vertical scaling

same as allway, work but was the cost worst it ?

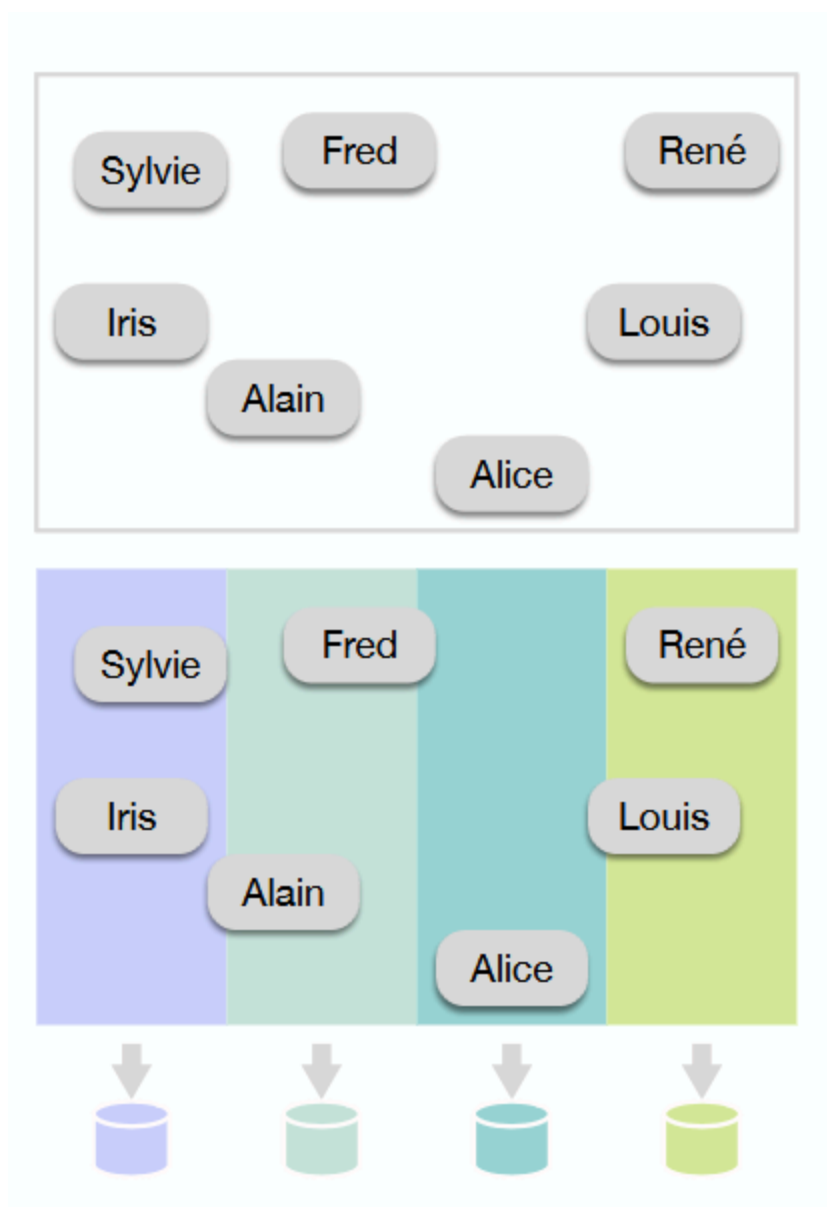
Horizontal scaling

Replication (single head) : just copy the entire thing else were



- one instance is the main one (leader) and the other replicate the main one (follower)
- To increase robustness, leader and followers should be in different availability zones.
- Database slow down a bit with synchronous replication

Partition : split the data base across a multitude of machin

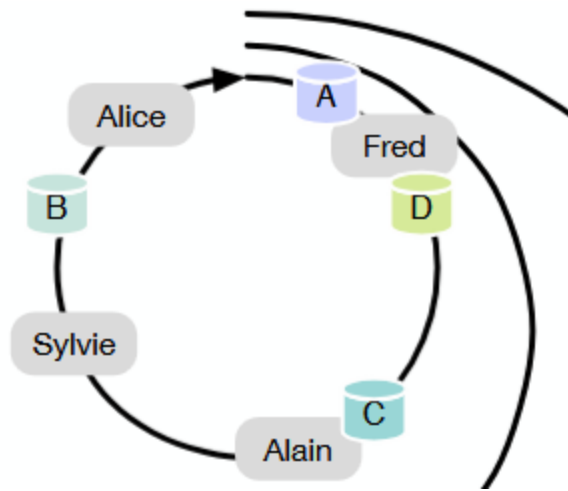


- En gros, a chaque requet faut passé par un "request router" qui vas rediriger la requet ver la bonne instance
- Les request router sont rapide et ne communique pas entre eux
- une hash fonction est utiliser pour determiner sur qu'elle instance vas chaque donner pour que se sois rapide a chercher (comme les bucket d'une hashmap)

problème : quand on ajoute une machine faut tous re-hash (comme quand on agrandi une hashmap)

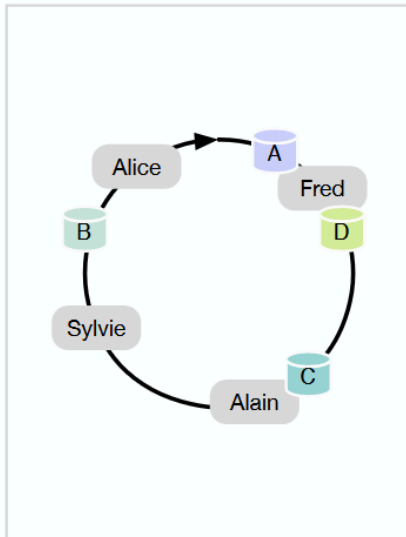
pour contré on peut utiliser une hash fonction consitente

- en gros, l'idée c'est de hash les value et de les assigner a une position sur un cercle.
- on hash aussi les machin (en utilisant leur nom comme key) et on les mette aussi sur le cercle
- les value sont placer sur la premier machine qu'il croise en continant clockwise

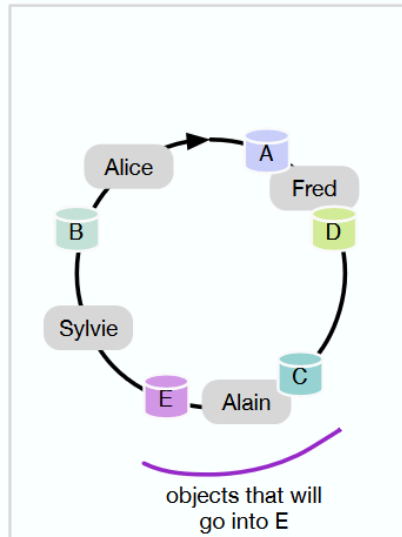


- celle evite de bouger tous quand on ajoute une machin

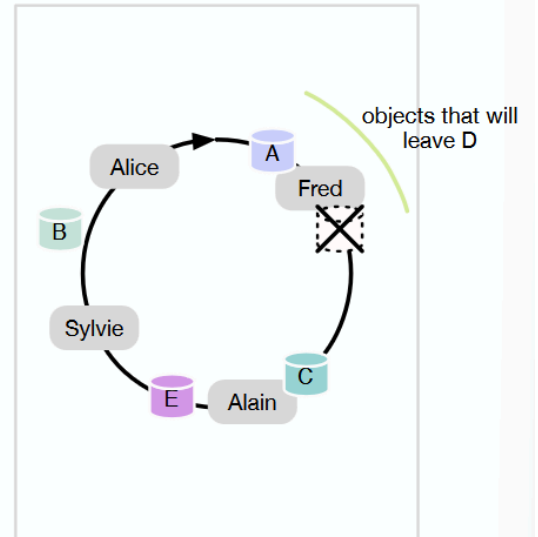
Initial situation



Adding a machine only affects the objects that are between the new machine and the preceding one.

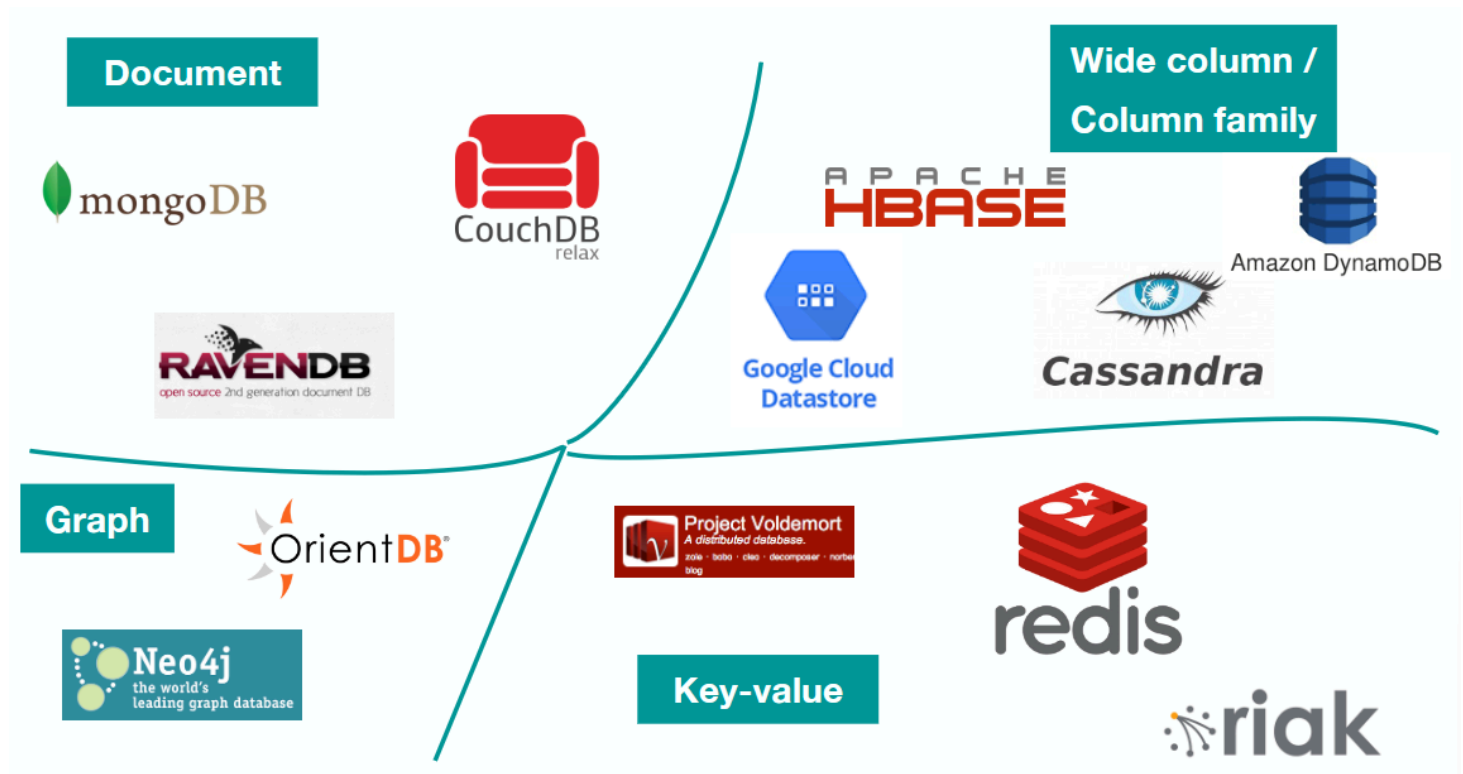


Removing a machine only affects the objects that are between the machine



problème : si une machine meurt la data base devien inutilisable
pour contré ça, on peut repliquer les donner des instance sur d'autre

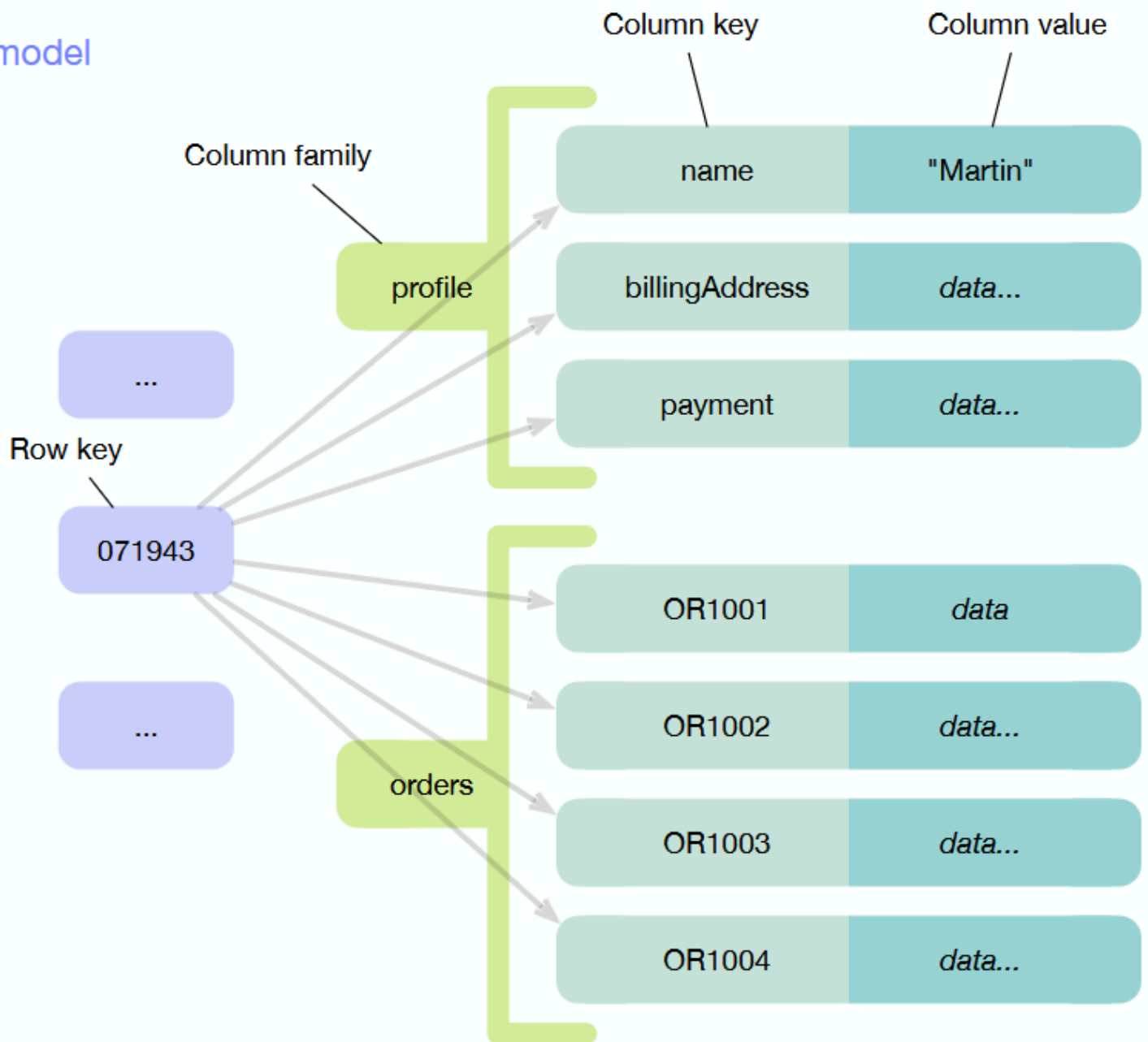
No SQL



Document model

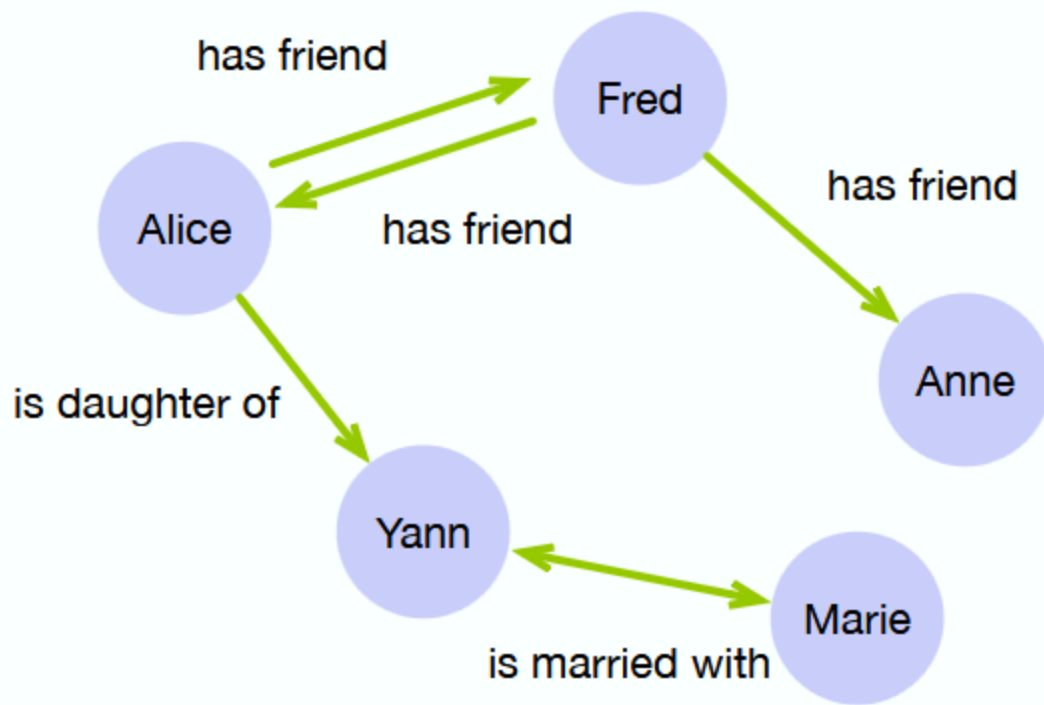
un peut style JSON si on veut

model



Graph model

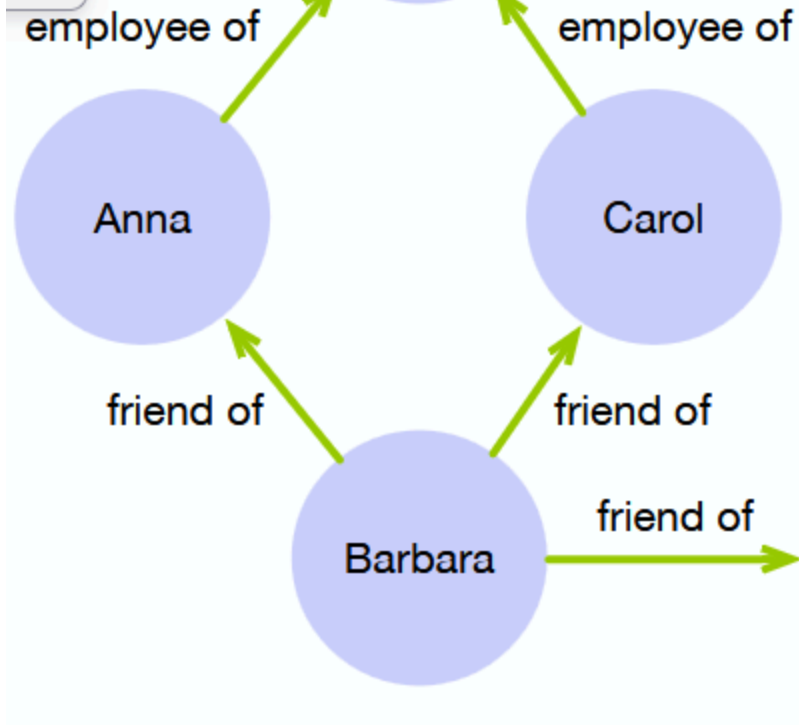
Cool pour les relation
en gros, bah tu fais un graph d'objet



```
START%barbara%=%node:nodeIndex(name%="Barbara")  
MATCH%(barbara)9[:FRIEND]9>(friend_node)  
RETURN%friend_node.name,friend_node.location
```

Barbara")

ion



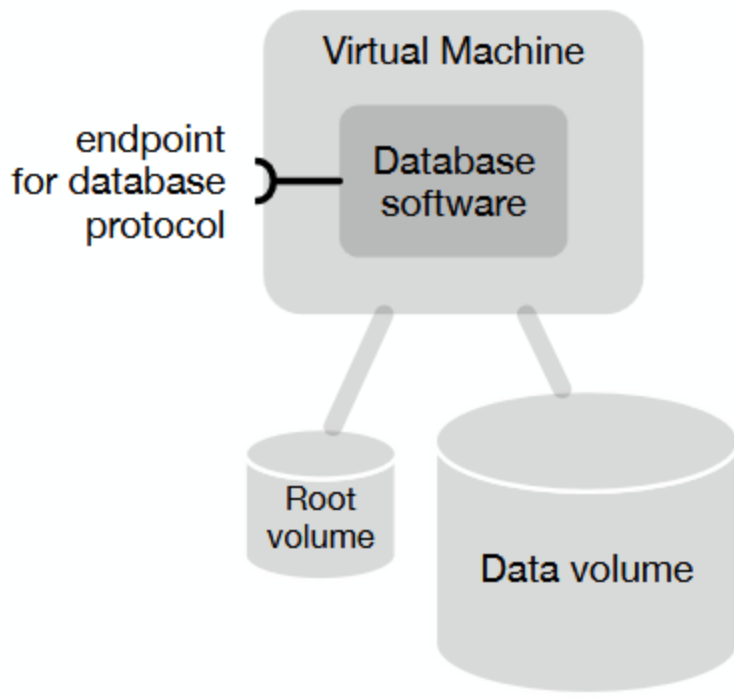
Cloud database (Database-as-a-Service)

En gros, c'est une database gérée par le fournisseur cloud

Single-tenant cloud databases

the database is dedicated to a single cloud client

basé souvent sur une techno existante



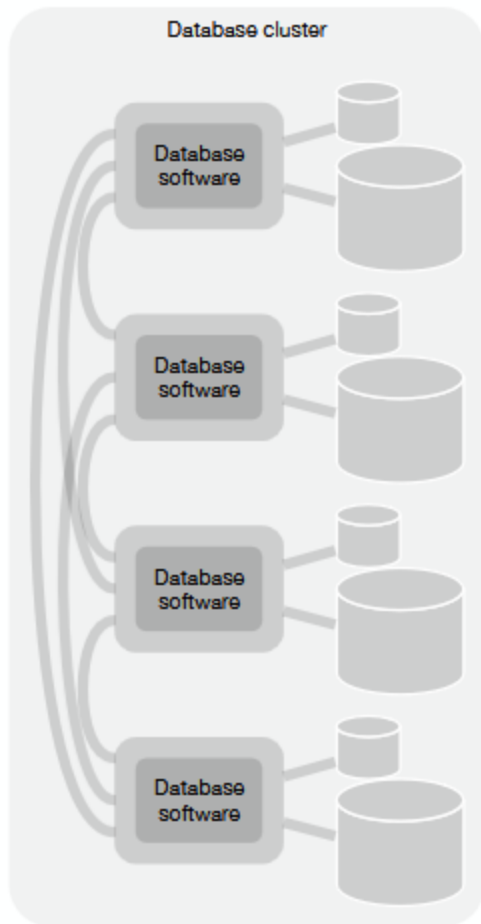
quand on a besoin d'un nouveaux volume, on crée un VM

Multi-tenant database

the database is shared between several cloud clients.

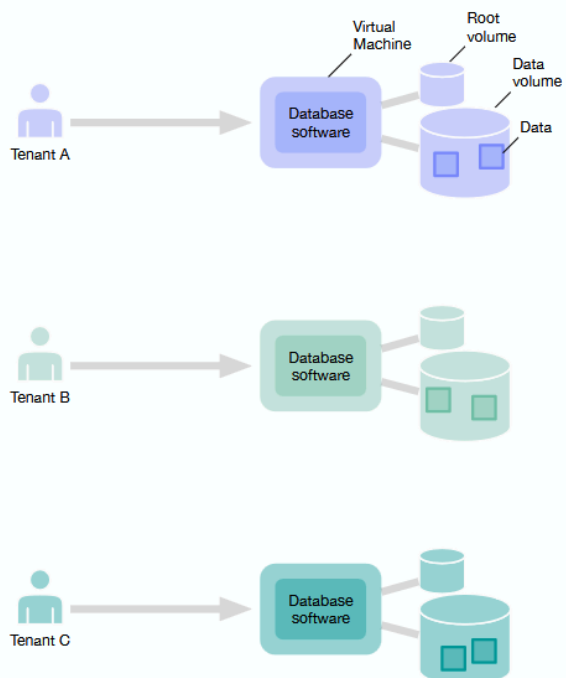
utilise souvent du NoSQL

Y a une database distribuer a travers plein d'instance

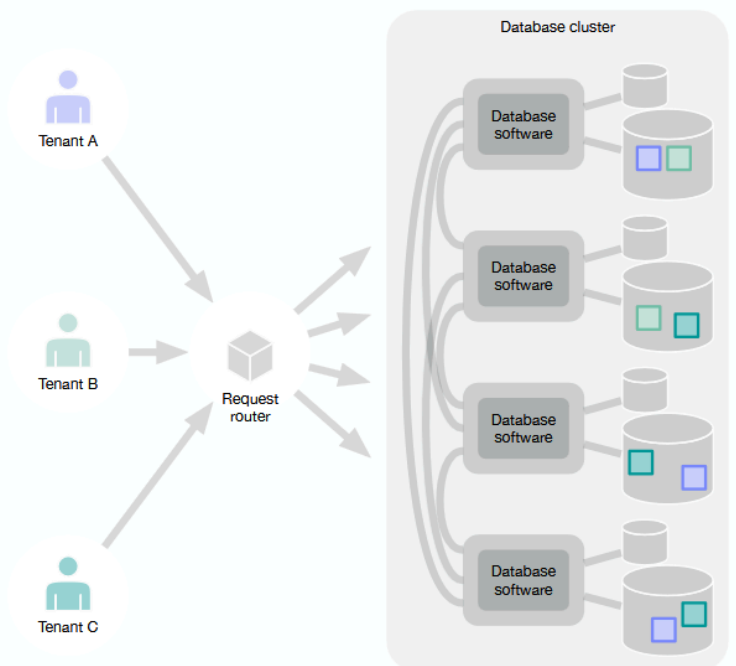


En gros

Single-tenant databases



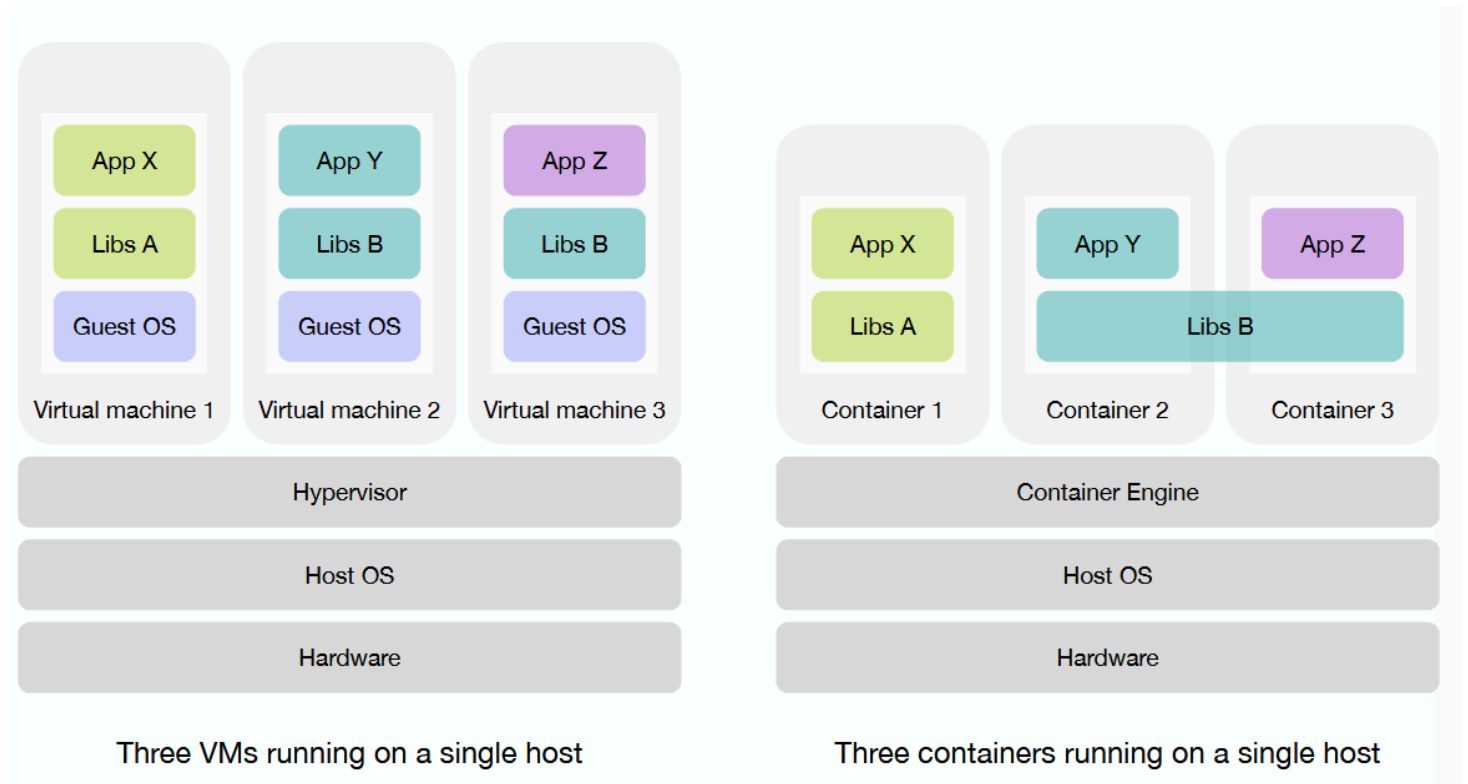
Multi-tenant database

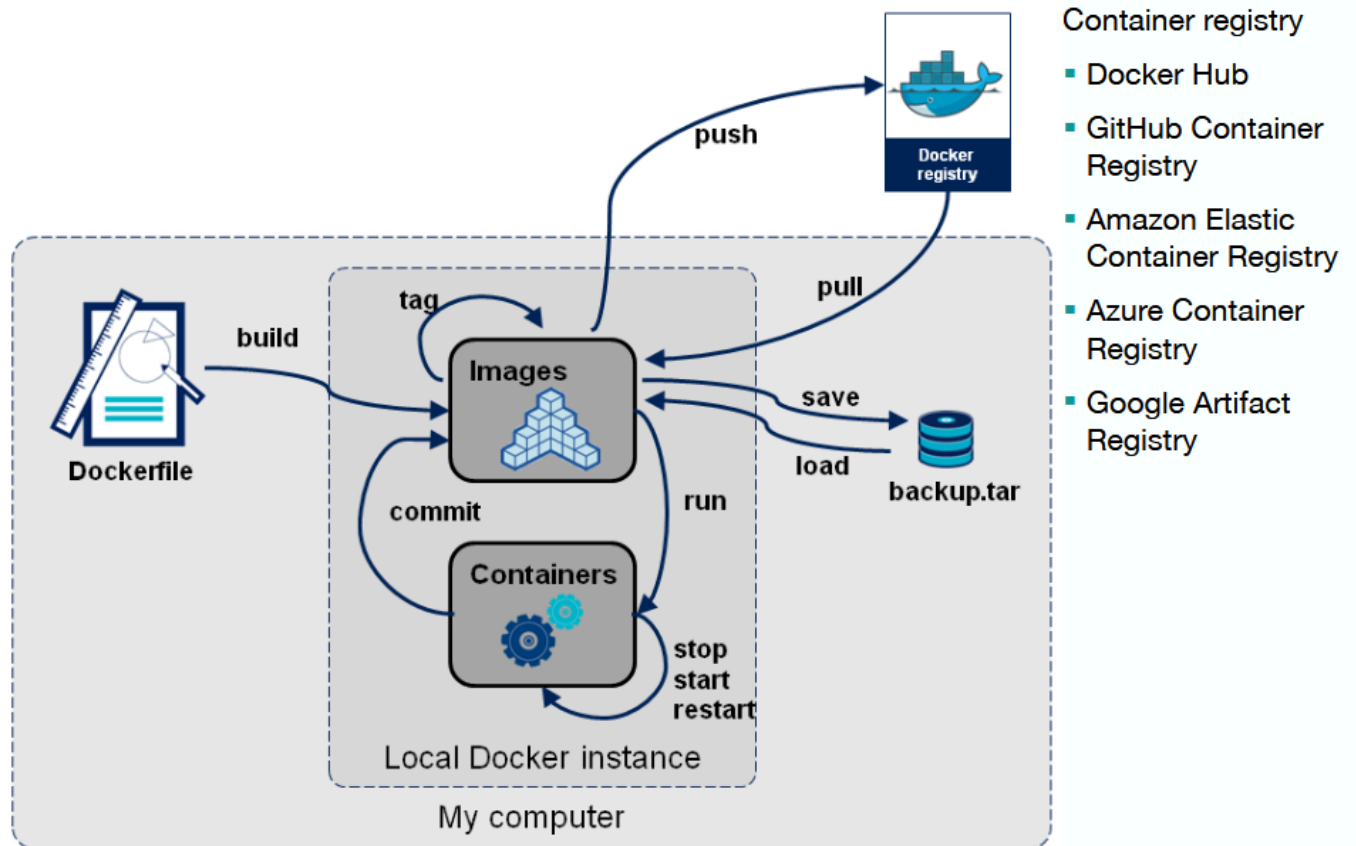


6 2/2 Container cluster management and orchestration

Container

En gros, c'est un moyen de contenir un process (avec un environnement defini) et de le run direct sur le l'OS host (un peu comme docker)





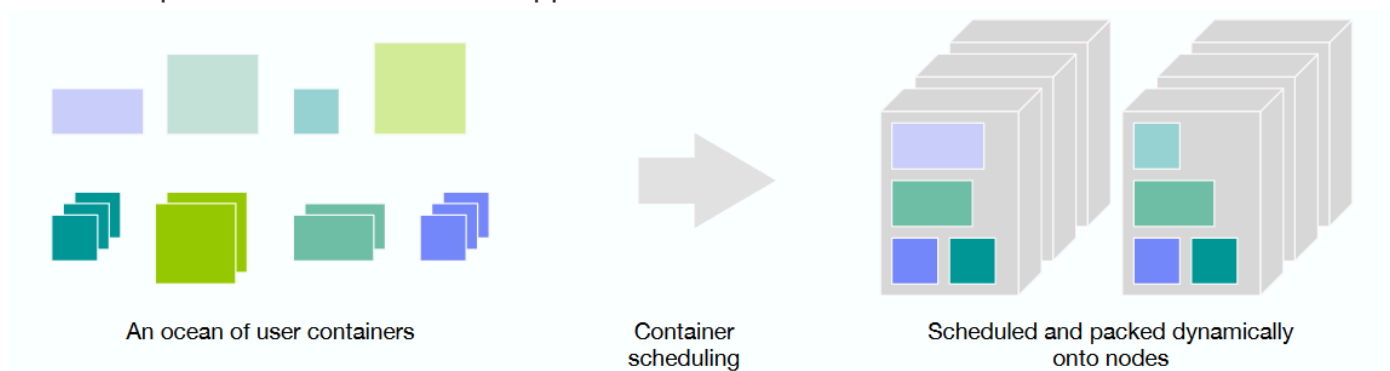
<https://docs.docker.com/en/docker-registry-first-steps/>

Cluster manager

C'est un peu comme un manager de container, ça permet, en plus de monitorer, de pouvoir créer des container sur d'autres machines tout en gardant la possibilité de créer des réseaux (style docker compose)

ça a comme but de :

- augmenter l'utilisation de cluster
- tout en respectant les contraintes des applications container



Kubernetes

C'est une plateforme pour automatiser :

- déployment
- scaling
- management of container app

Kubernetes VS IaaS

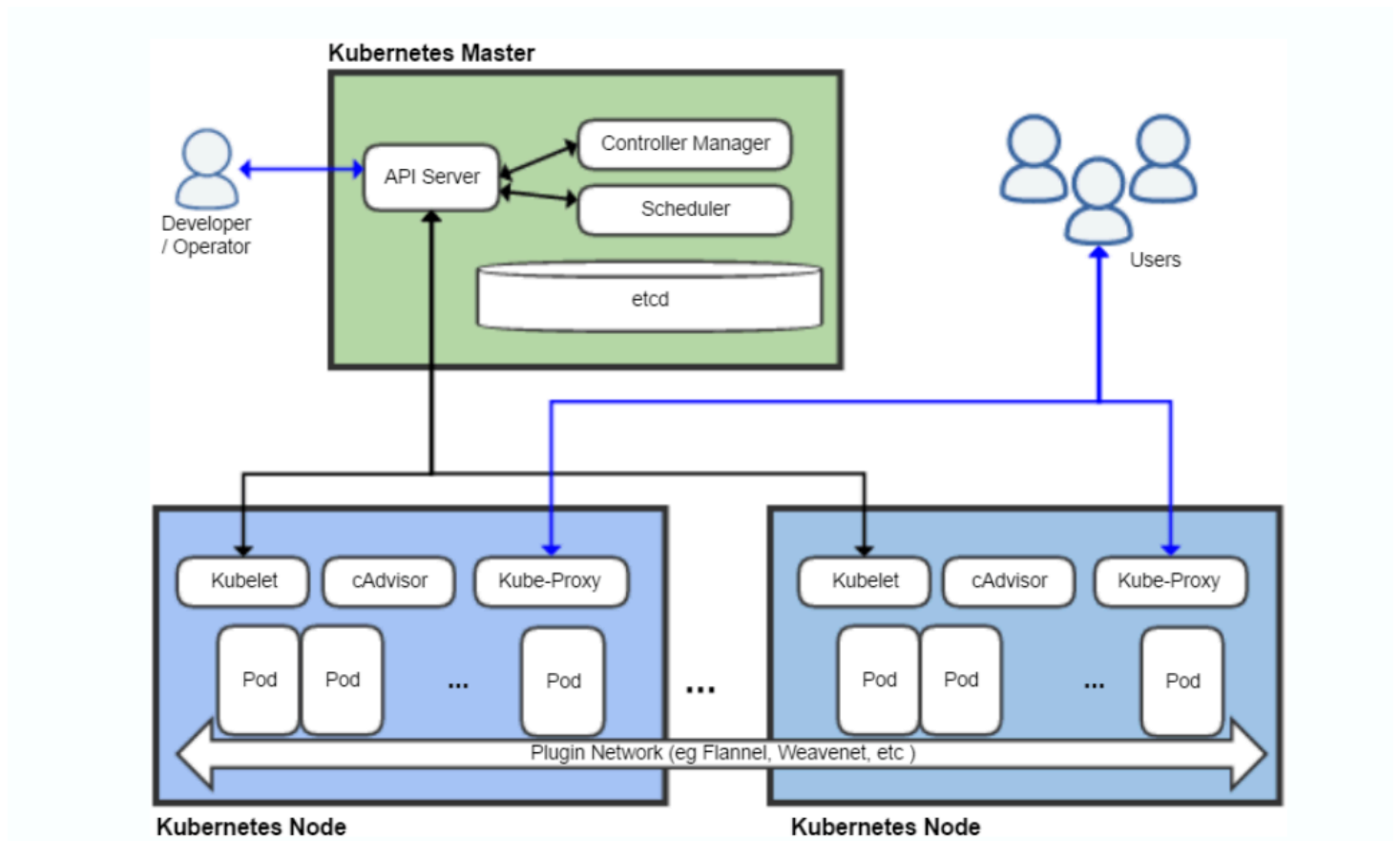
IaaS

- Launch a virtual machine.
- Connect to the VM via SSH.
- Install application on VM.
- Create VM image from VM.
- Create load balancer.
- To launch more VMs from VM image create a Launch Template and an Autoscaling Group.
- Connect VMs to load balancer.

Kubernetes

- On local machine, create container image for app using Docker and Dockerfile.
- Upload container image to a container registry.
- Write a Kubernetes **manifest** file in YAML that describes a **Pod** that will contain your container.
- Launch the Pod (with the container) using the command line:
`kubectl create -f pod.yaml`
- To launch multiple replicas of your container, write a **Deployment** manifest.
- For load balancing, write a **Service** manifest.

Cluster / POD logic



en gros, le travail est repartie dans des POD qui peuvent être crée et detruit facilement, le Kurbernets Master ce charge des les grée.

- ETCD : c'est {Key,Value} store qui permette de configurer les cluster, ça représent l'état global des cluster
- API Server : API qui permmet manage pour les devops
- Scheduler : selection qu'elle node devrais run ou non suivant les ressources
- Controller Manager : C'est le process principal qui permette de controller tous le truc

Definition

Cluster

Ensemble de machin ou les Pods sont déployer

Pod

1 à n container qui sont garentie de devoir être ensemble (genre backend et database) sur la meme machine

Controller

C'est un process qui permet au cluster d'aller vers l'état demandé par le master

Service

Ensemble de Pod qui marche ensemble

Label

c'est just un tag

Common concepts

Kubernetes a un objet API

command pour crée objet `kubectl create -f file.yaml`

YAML

exampe :

```
apiVersion: v1
kind: Pod
metadata:
  name: redis
  labels:
    component: redis
    app: todo
spec:
  containers:
  - name: redis
    image: redis
    ports:
      - containerPort: 6379
    resources:
      limits:
        cpu: 100m
    args:
      - redis-server
      - --requirepass ccp2
      - --appendonly yes
```

Object

un objet est composé dans l'ordre de :

- api version
- kind
- d'une struct metadata (info de l'objet)
- d'une struct spec (targeted state de l'objet)

Quand Kurb crée un objet, il ajoute des donnée sur sont yaml :

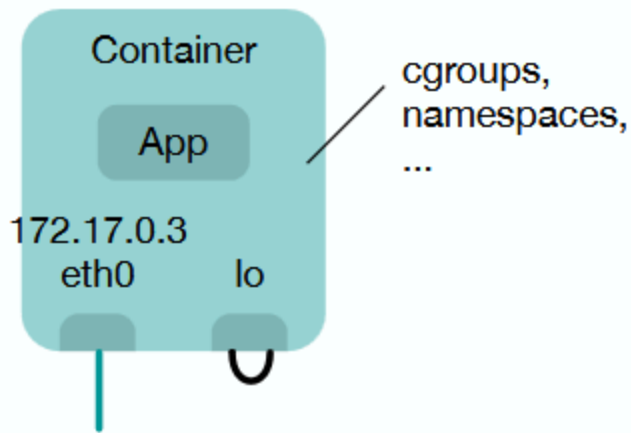
dans le metadata

- uid de l'objet
- ressourceVersion de l'objet

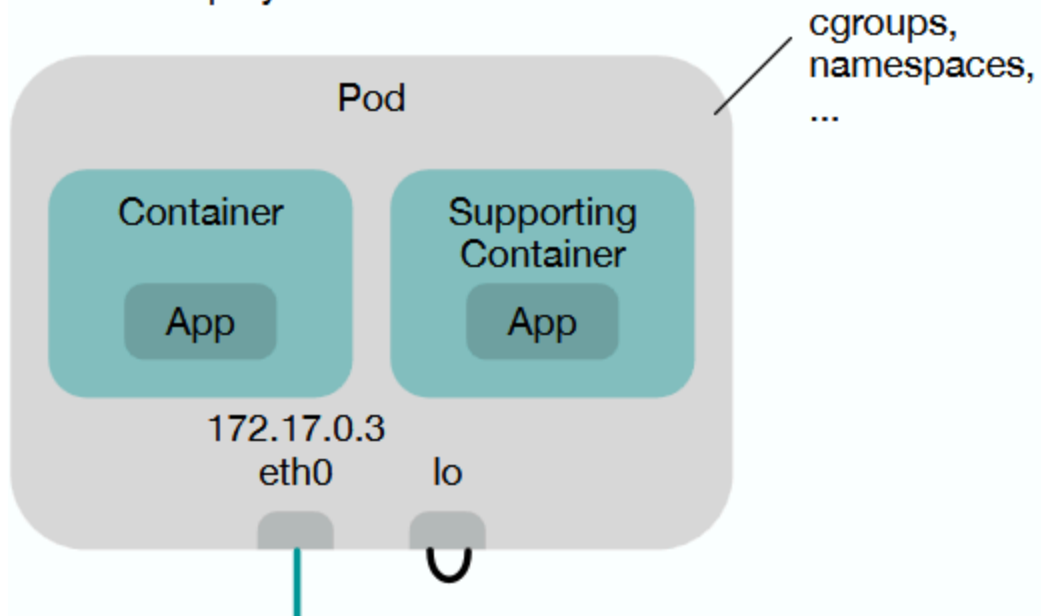
et aussi ajoute la struct **status** qui à des info sur le state actuel de l'objet (en read only)

Pod

Unit of deployment in Docker: Container



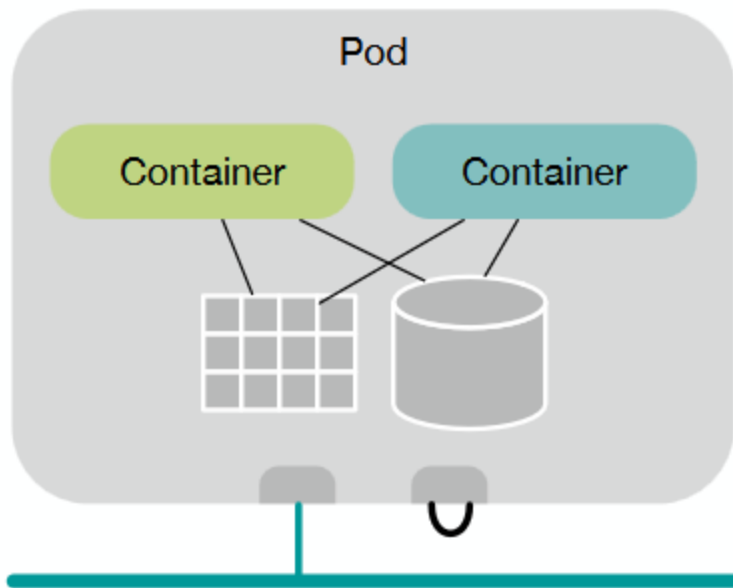
Unit of deployment in Kubernetes: Pod



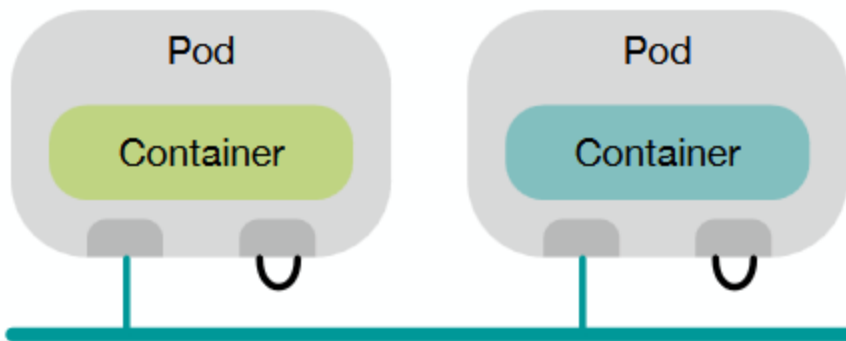
vue que les pod c'est une abstraction, Kurb nous simplifie la vie en nous permettant de les config de 2 manière :

- loosely coupled : chaque container a son pod
- tightly coupled : un pod pour tous les container concerné

Tightly-coupled containers



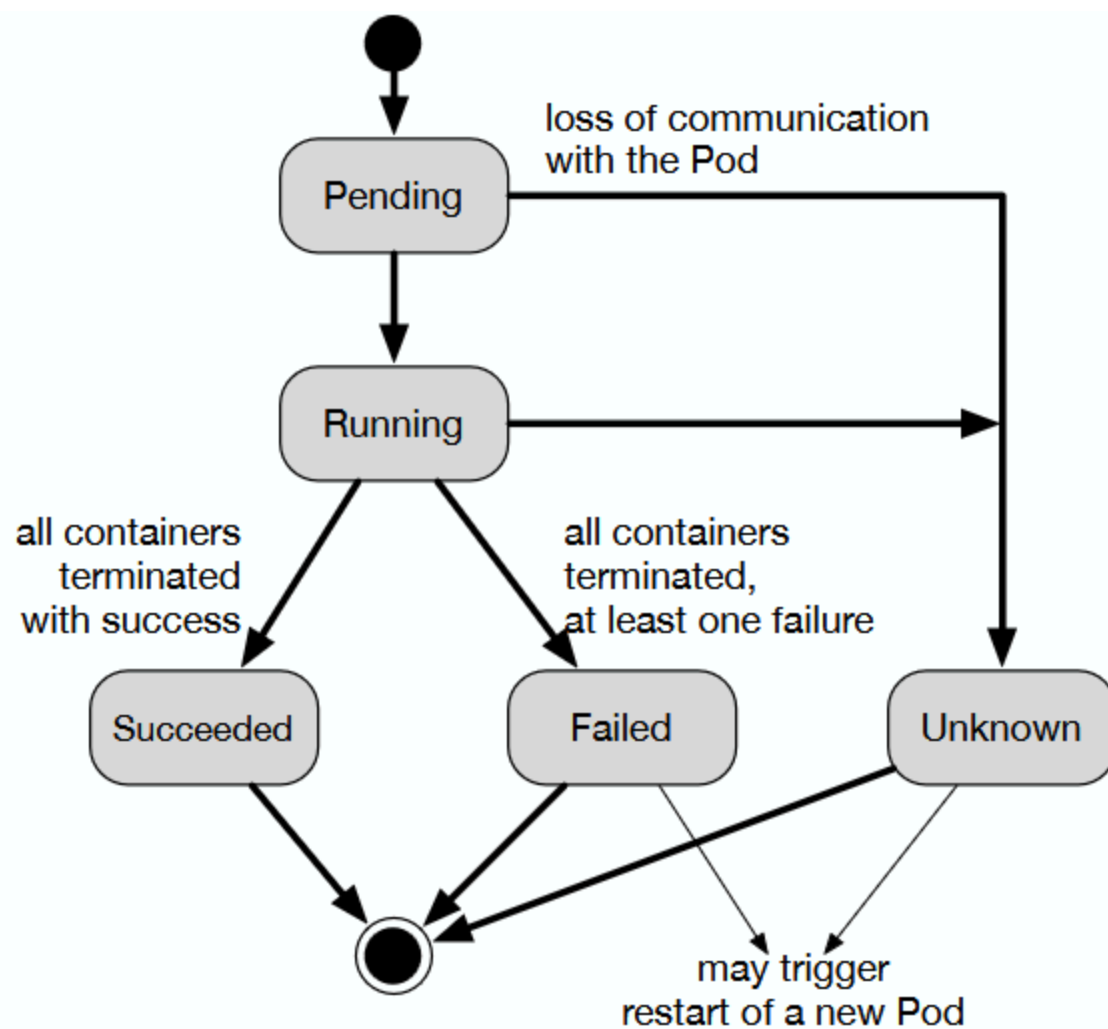
Loosely-coupled containers



Deployment

Les pod sont sencé être de la plus petit unité de scaling (genre en gros, c'est ton container mais au plus simple)

Déployer un pod ne coute pas chère et donc, quand un pod meurt ou crache, on en recrée just un nouveau



Definition

```
apiVersion: v1
kind: Pod
metadata:
  name: www
spec:
  containers:
  - name: nginx
    image: nginx
    volumeMounts:
    - mountPath: /srv/www
      name: www-data
      readOnly: true
  - name: git-monitor
    image: kubernetes/git-monitor
    env:
    - name: GIT_REPO
      value: http://github.com/some/repo.git
    volumeMounts:
    - mountPath: /data
      name: www-data
  volumes:
  - name: www-data
    emptyDir: {}
```

Name of Pod

Internal name of the container
in the Pod

Container image name

Environment variable name
and value for the container
(docker run -e
GIT_REPO=... ..)

Volume name and type

Command

- déclarer un pod : `kubectl create -f docs/user-guide/walkthrough/pod-nginx.yaml`
- list les pod : `kubectl get pods`
- kill un pod : `kubectl delete pod nginx`

Environment variable

On peut déclarer des variables d'environnement en ajoutant la struct suivante à **spec** :

```
env:
- name: REDIS_ENDPOINT
  value: redis-svc
- name: REDIS_PWD
  value: ccp2
```

Aussi Kubernetes crée par défaut les variables suivantes :

- HOME : as the default user within a container is root, the default homedir is /

- HOSTNAME : hostname associated with the container
- PATH : by default includes /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
- TERM : xterm if the container is allocated a pseudo terminal entry point (interactive mode only)

Volume

Les Pod peuvent avoir des volumes. C'est un system de fichier que les app peuvent lire et écrire

Les volume sont partager a travert les container du meme pod

Si les volume ne sont explicitment dit comme **persistant** alors ils meurt avec les pod

Type de volume

- emptyDir : empty volume | del with pod
- hostPath : path sur la machin host | persite sur la machine
- secret : volume crypter
- NFS : network file system | persite

```
kind: Pod
apiVersion: v1
metadata:
  name: mypod
spec:
  containers:
    - name: myfrontend
      image: dockerfile/nginx
      volumeMounts:
        - mountPath: "/var/www/html"
          name: mypd
  volumes:
    - name: mypd
      persistentVolumeClaim:
        claimName: myclaim
```

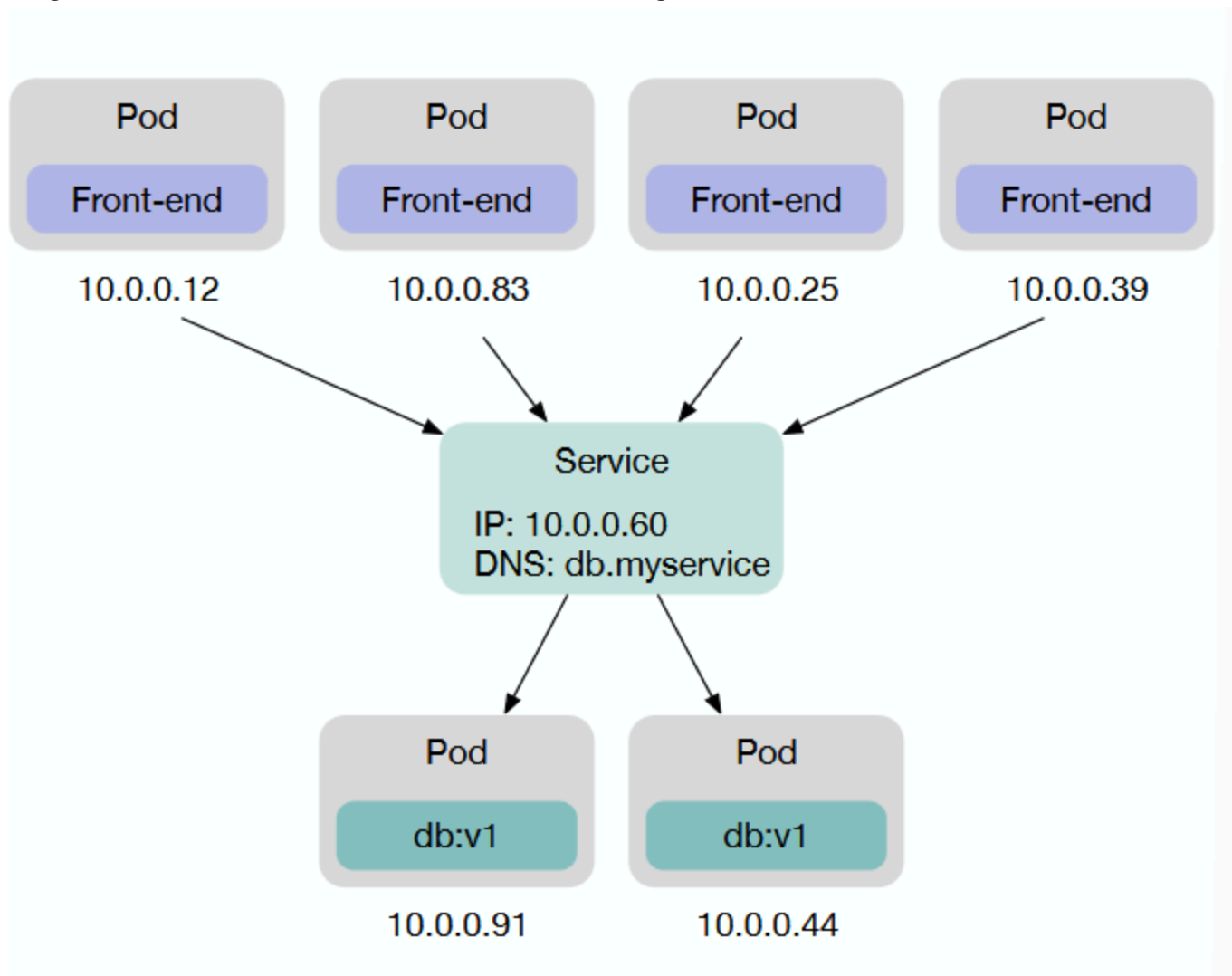
```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: myclaim
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 8Gi
  # The cluster must have
  # a configured storage
  # class
  storageClassName: default
```

Replication Controller

Permette de déployer facilment les pod. En gros, il prend la def d'un pod et il en crée autant que besoin

Service

En gros, c'est un proxy, vu que les pod peuvent être détruit et changer rapidement, ça nous permet de garder la même adresse IP sans avoir à la changer.



Type de service

- ClusterIP : utiliser que dans le cluster pour gérer ses IP internes
- NodePort : en plus du ClusterIP permet d'exposer un port vers les autres nœuds
- LoadBalancer : en plus du NodePort permet de faire office de load balancer

```
apiVersion: v1
kind: Service
metadata:
  name: zurmo-apache
spec:
  ports:
    - port: 80
      protocol: TCP
      targetPort: 80
  selector:
    name: apache.zurmo
  type: LoadBalancer
```

Label

Un service peut faire le filtre des pod grace au label

Deployments

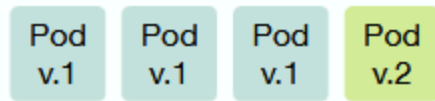
En gros, c'est une task qui vas lentement repliquer des pod avec la nouvel version puit vas detruire les ancien pods, si quelque chose se passe male, ça vas rollback

Desired state:
3 Pods using
image v.1

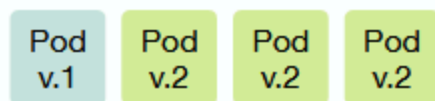
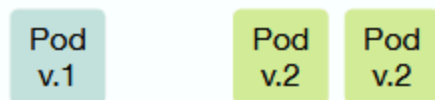
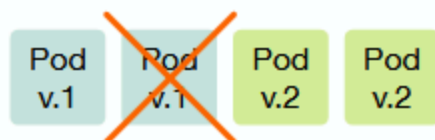
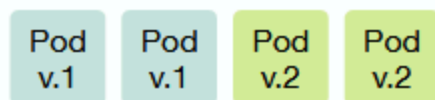
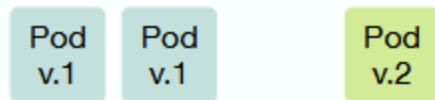


Deployment
complete

Change of
desired state:
use image v.2



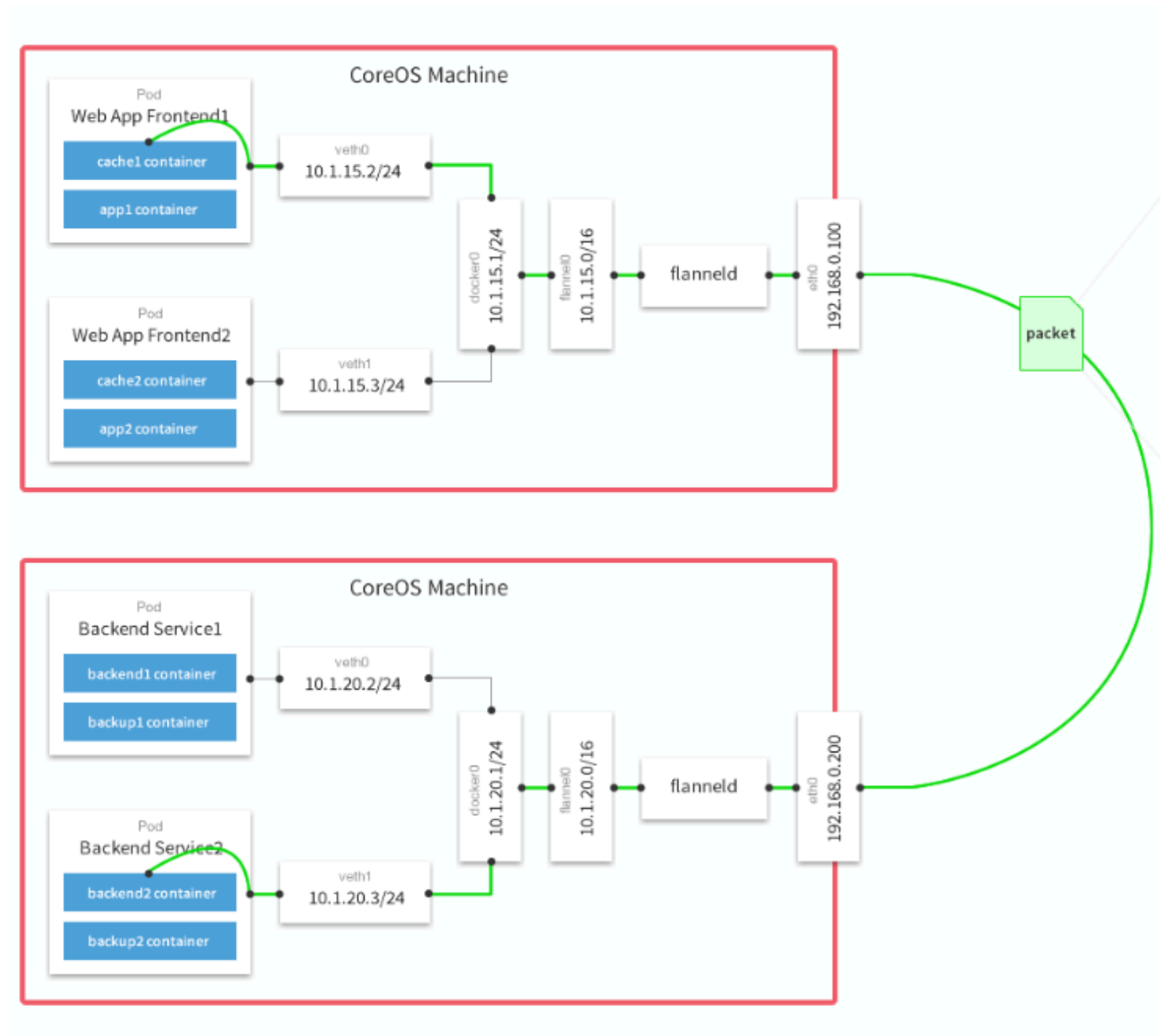
Deployment
progressing



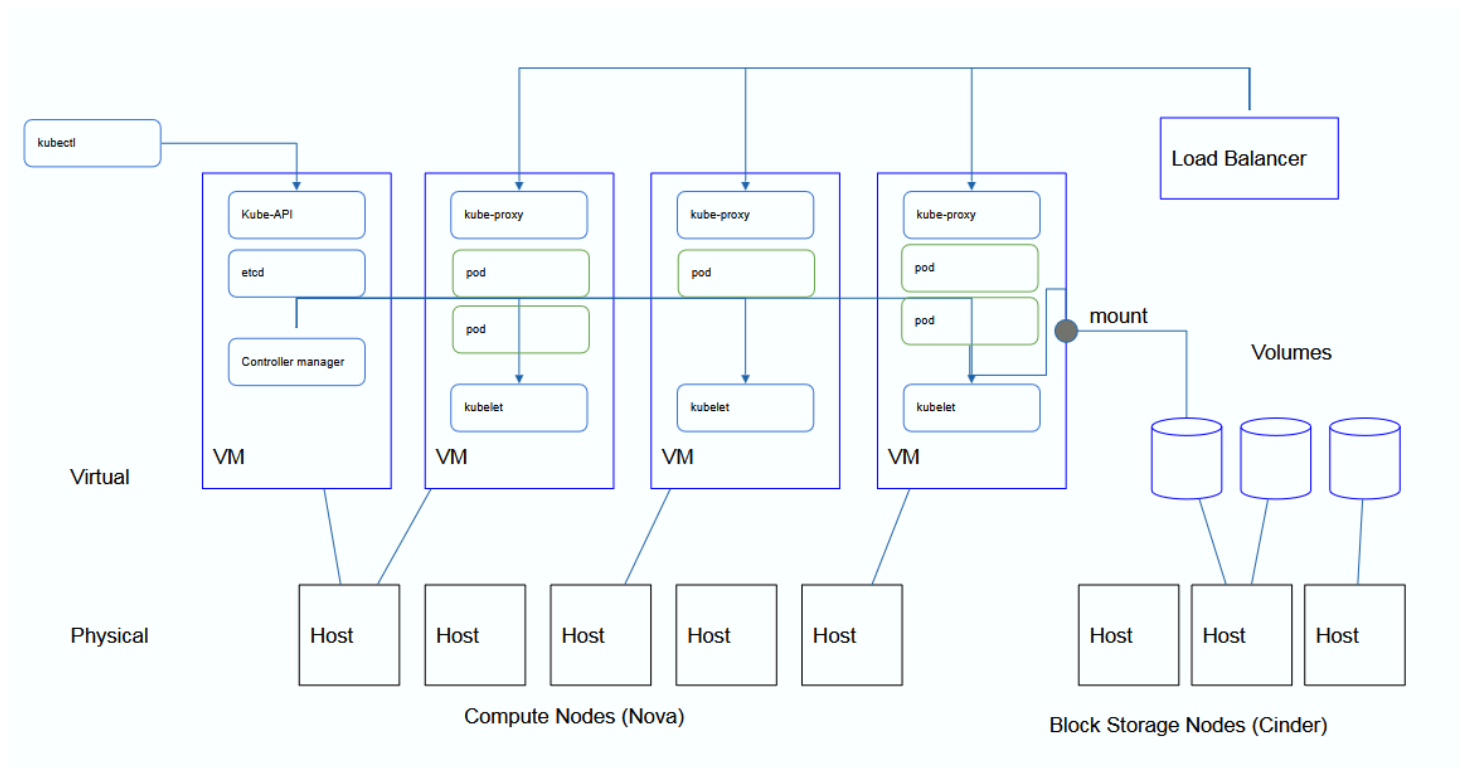
Deployment
complete

c'est géré par update (rolling update)

Multi-host networking

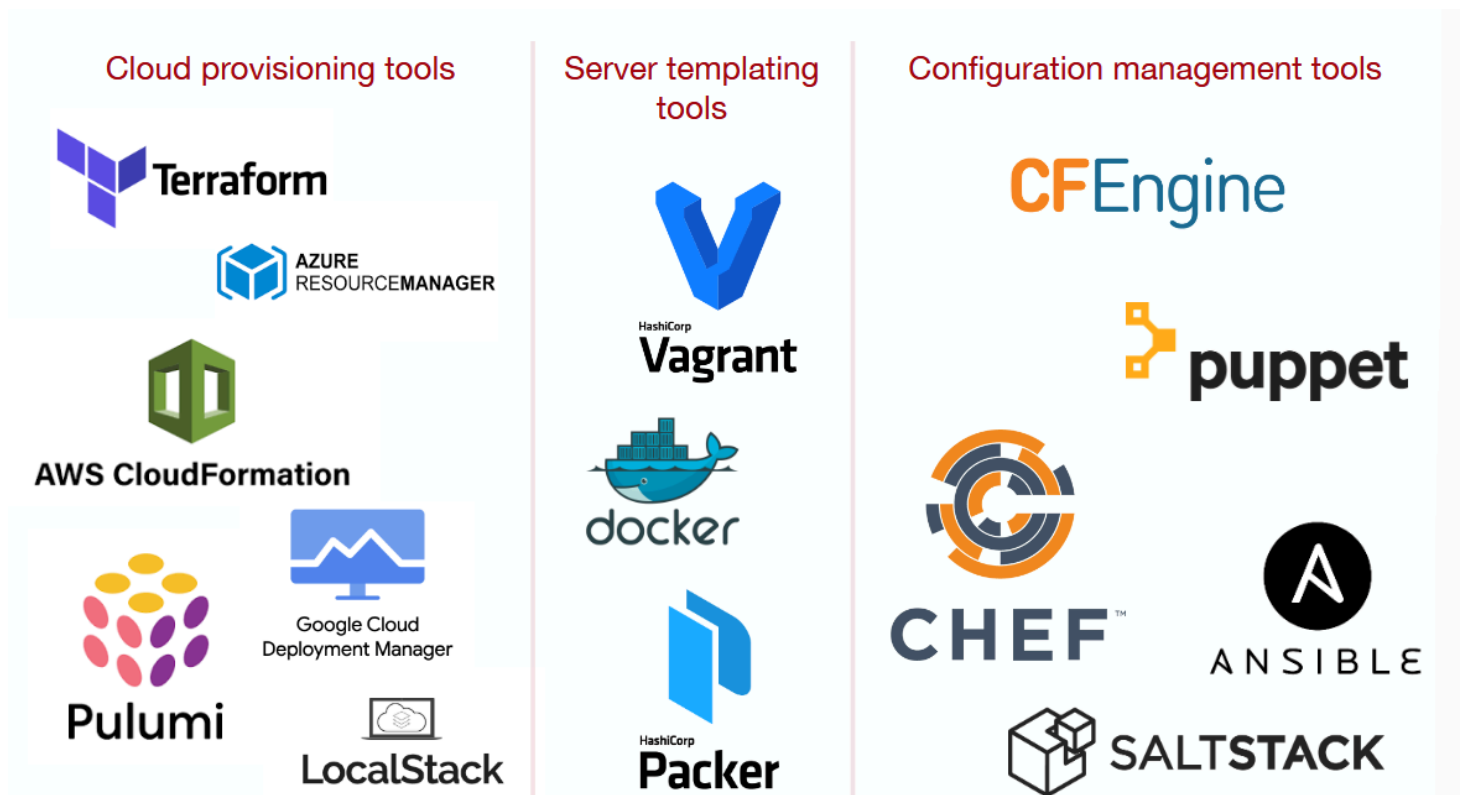


Kurb on IaaS

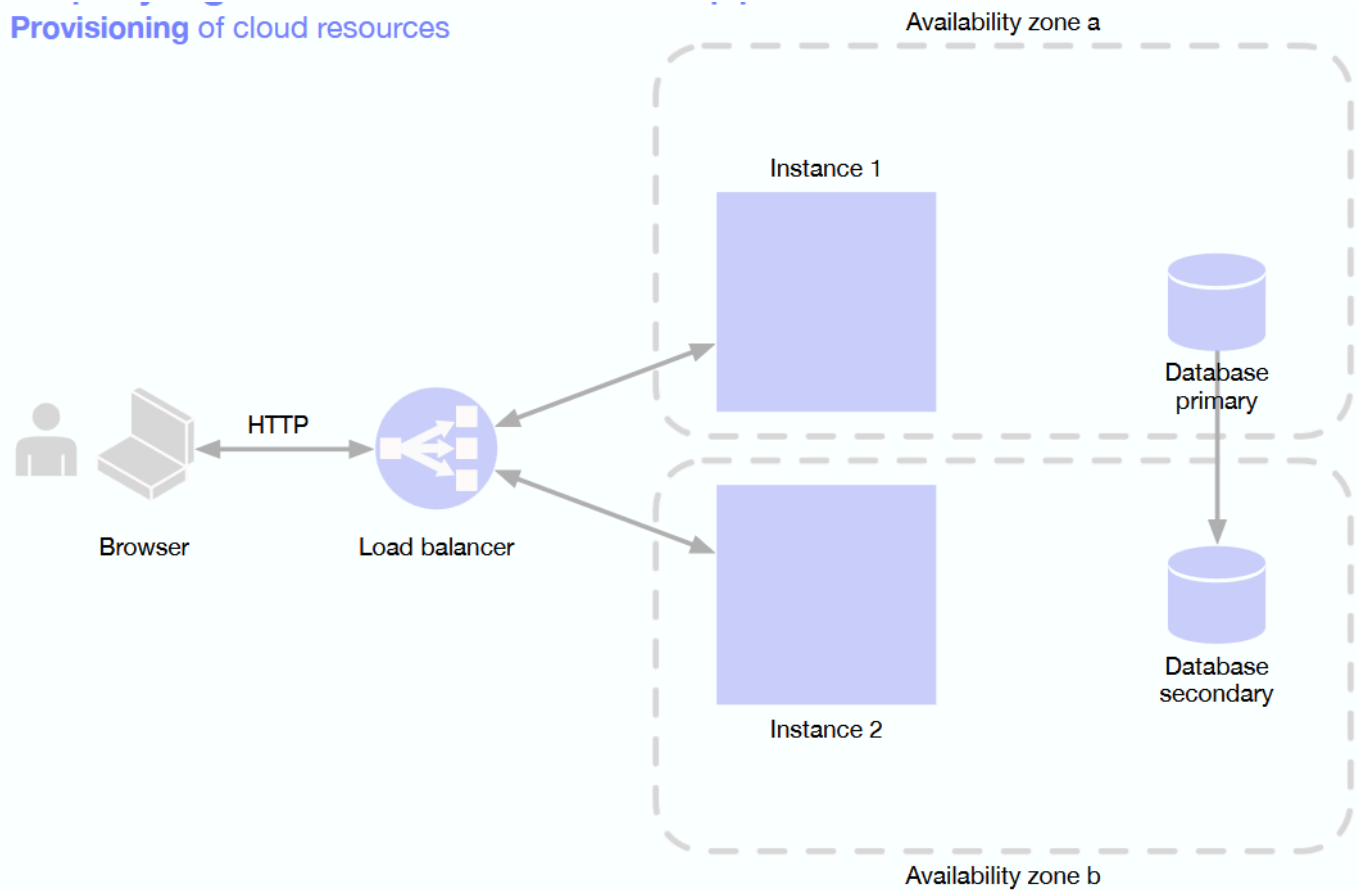


7 Infrastructure as Code (IaC)

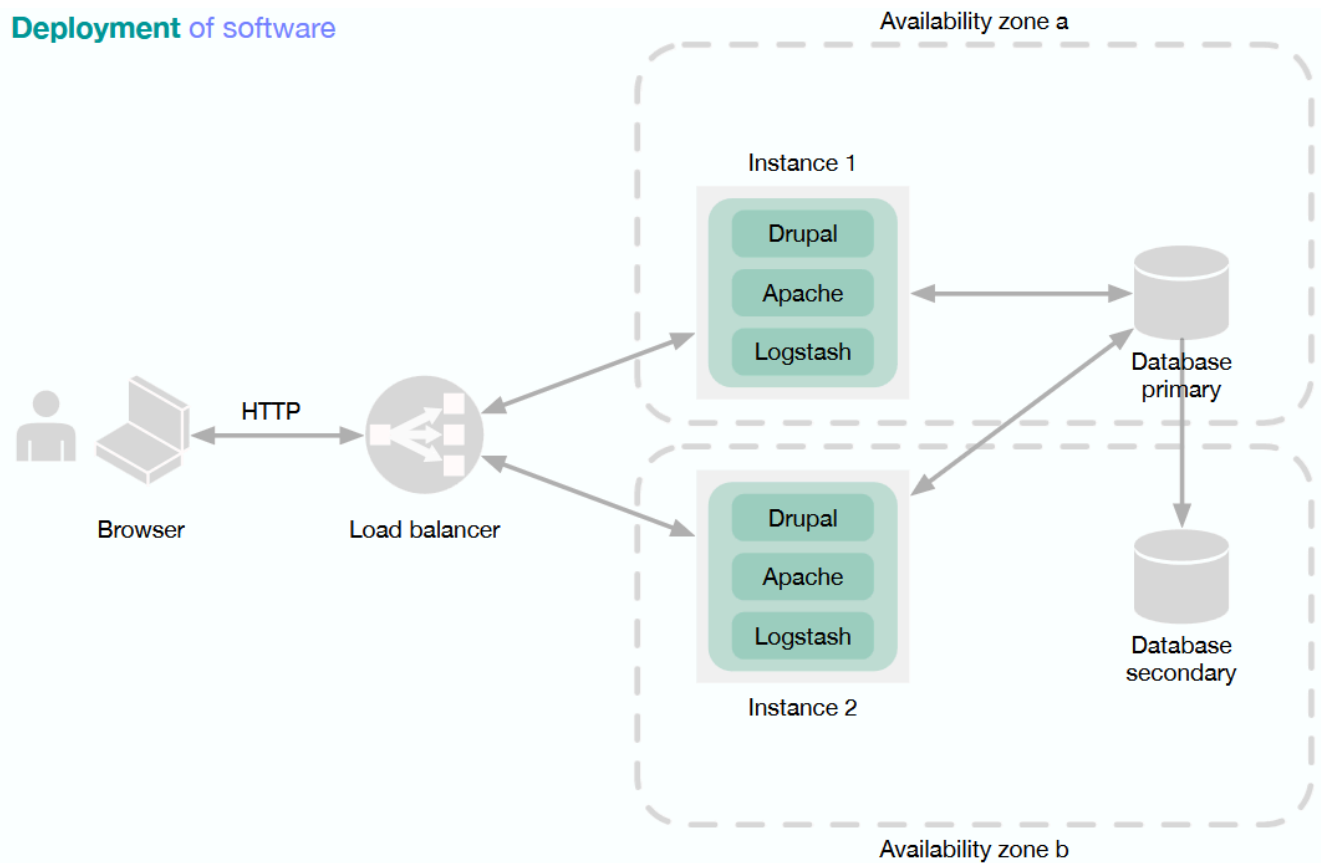
L'idée est de décrire une infra avec des **definition files**



Provisioning of cloud resources



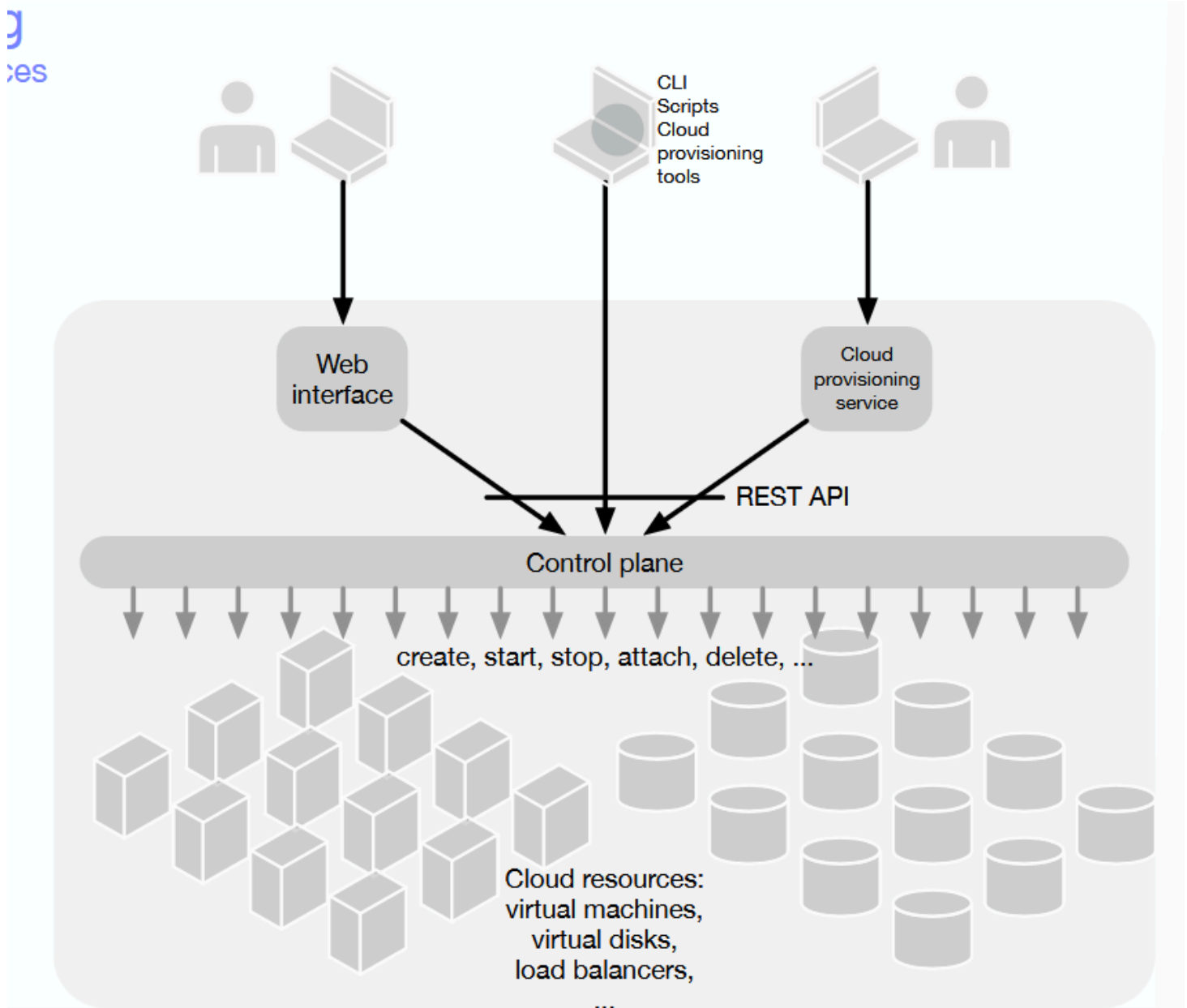
Deployment of software



Cloud provisioning

c'est un outil pour choper des ressources cloud (marche avec une API)

Utilise une interface Web ou un CLI



AWS cloudFormation

```
AWSTemplateFormatVersion: "2010-09-09"
Description: A sample template
Resources:
  MyEC2Instance:
    Type: "AWS::EC2::Instance"
    Properties:
      ImageId: "ami-0ff8a91507f77f867"
      InstanceType: t2.micro
      KeyName: testkey
      BlockDeviceMappings:
        -
          DeviceName: /dev/sdm
          Ebs:
            VolumeType: io1
            Iops: 200
            DeleteOnTermination: false
            VolumeSize: 20
  MyEIP:
    Type: AWS::EC2::EIP
    Properties:
      InstanceId: !Ref MyEC2Instance
```

- template : un text file qui contient la desc de la ressource cloud
- Stack : list des ressource unit utiliser par le cloud (typiquement là ou ce retrouve les liste des vm utiliser pour loadbalancing)
- Change set : en gros, c'est un sommaire des changement a apporter au ressource déjà en route

Terraform

C'est un outil qui permet faire du IaC opensource

Concept

Config file

décrit la ressource cloud

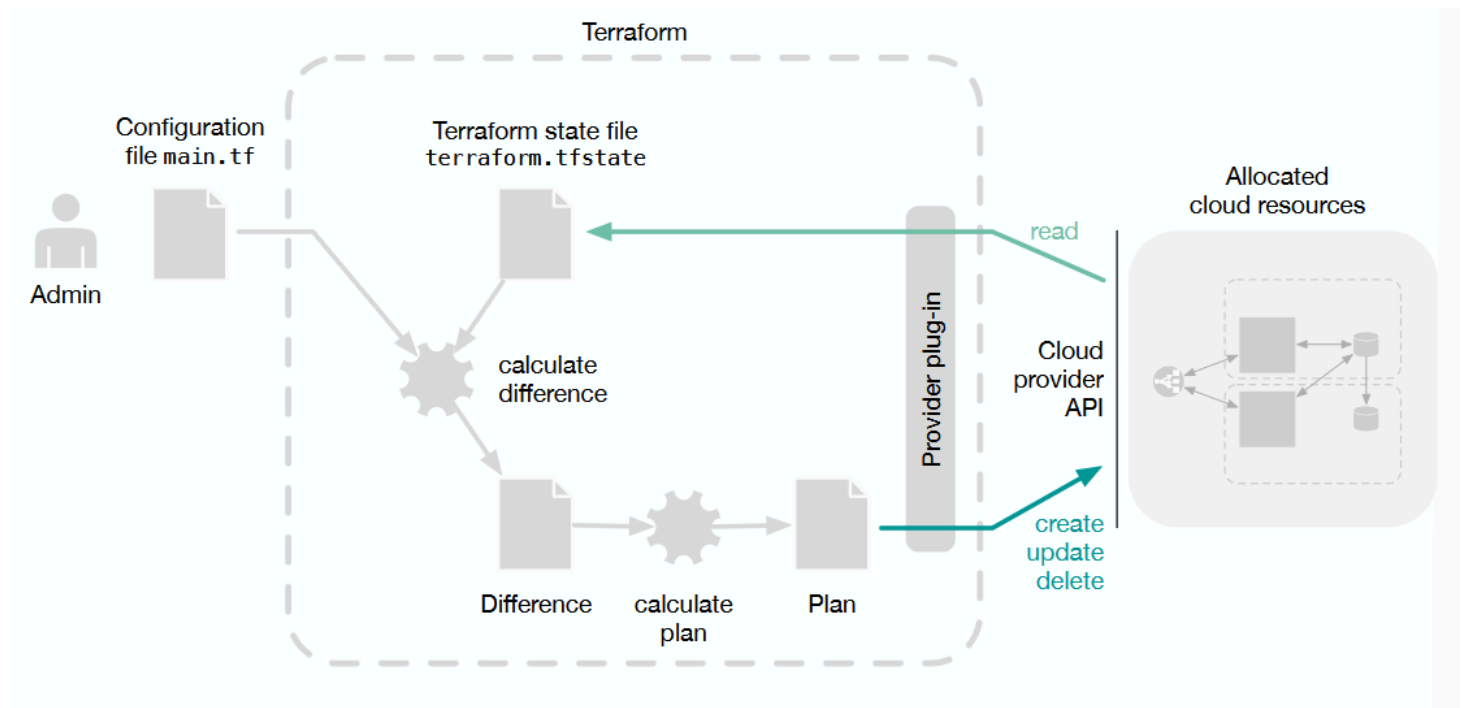
Terraform peut géré des composent de bas niveau

Execution plan

Terraform generates an execution plan describing what it will do to reach the desired state

Resource graph

Terraform builds a graph of all resources, and parallelizes the creation and modification of any non-dependent resources



Principle of Desired State Configuration

en gros, l'idée c'est que le devops donne un state que les instances doivent atteindre, les outiles determines les difference entre le state targeted et le state actuel, depuis ses difference, les outile quesqu'il faut changer et l'execute vier un a reconciliation loop

Common command

- `terraform init` : Prepares the working directory for other commands.
 - Creates `.terraform` directory to store plugins, the currently active workspace, the last known backend configuration, etc.
 - Downloads plugins
 - Accesses state in the backend
- `terraform validate` : Checks whether the configuration is valid.
- `terraform plan` : Shows changes required by the current configuration without executing them.
- `terraform apply` : Executes changes to update infrastructure or creates infrastructure if it does not exist yet.
- `terraform destroy` : Destroys previously-created infrastructure.

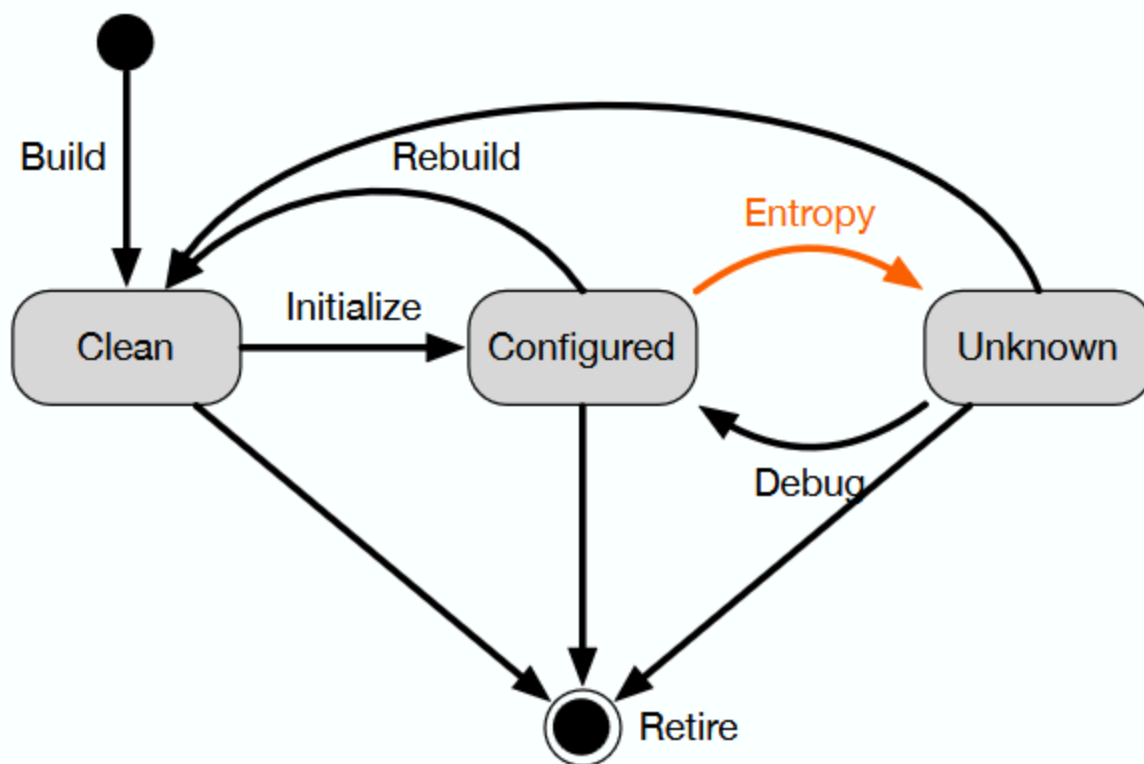
Example

```
provider "aws" {  
  region = "us-east-2"  
}  
  
resource "aws_instance" "example" {  
  ami = "ami-0c55b159cbfafa1f0"  
  instance_type = "t2.micro"  
  
  tags = {  
    Name = "terraform-example"  
  }  
}
```

Configuration management

The problem of entropy

Machine life cycle



the problème

manual configuration :

- too easy to make mistakes
- changes not guaranteed to be reproducible
- not simple to scale
- no auto documented

Shell script :

- script assume the instance is in a certain state, if not, were fucked
- if script break in the middle of a execution, were fucked
- unix commands have very different set of options depending on the OS

Desired-State Configuration

En gros, l'idée c'est qu'à la place de dire à un outil ce que l'on veut qu'il fasse, on lui dit où on veut aller (déclarative au lieu d'imperative)

Idempotent commands

c'est des commandes qui marchent par rapport au state visé

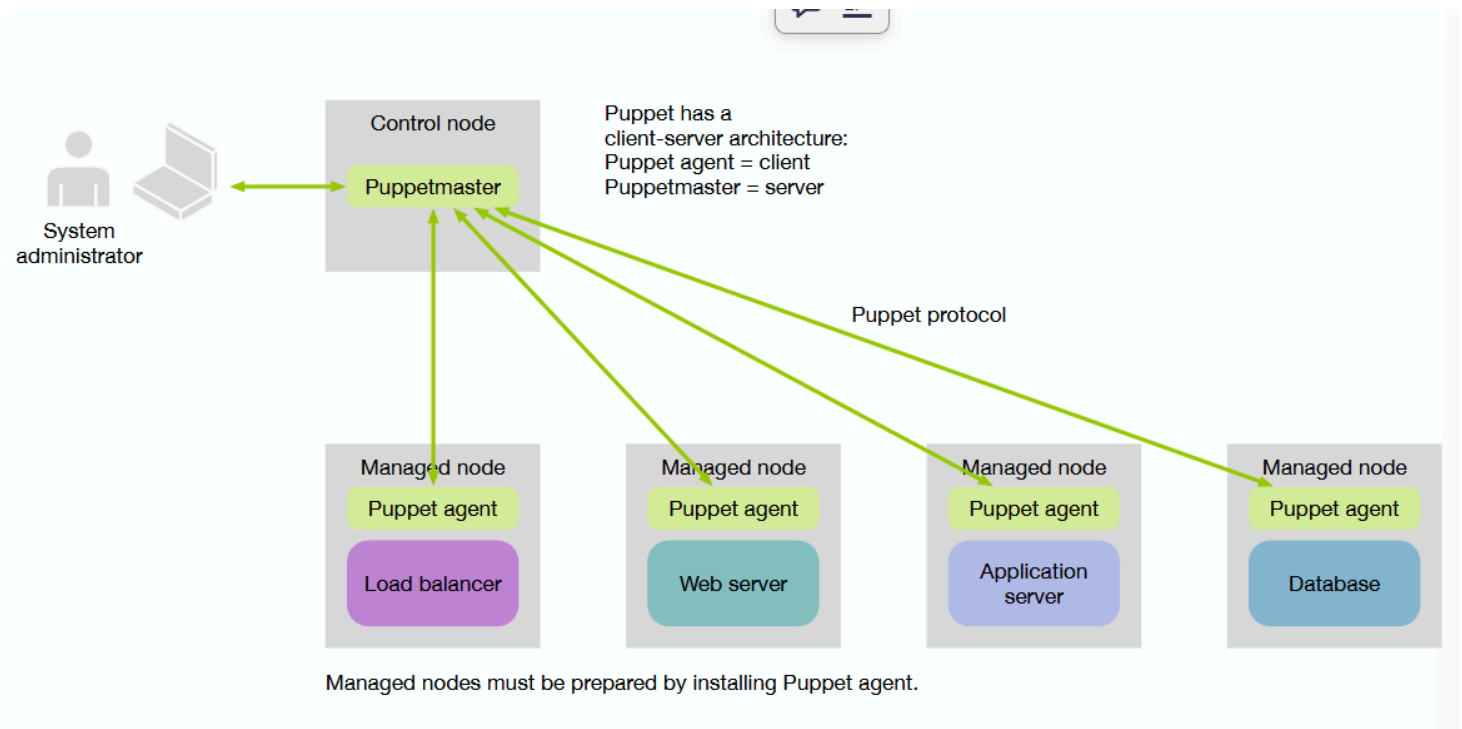
exemple : ssh-keygen, 1st run = create new key, 2nd run = overwrite old key with new ones

donc, Idempotent command :

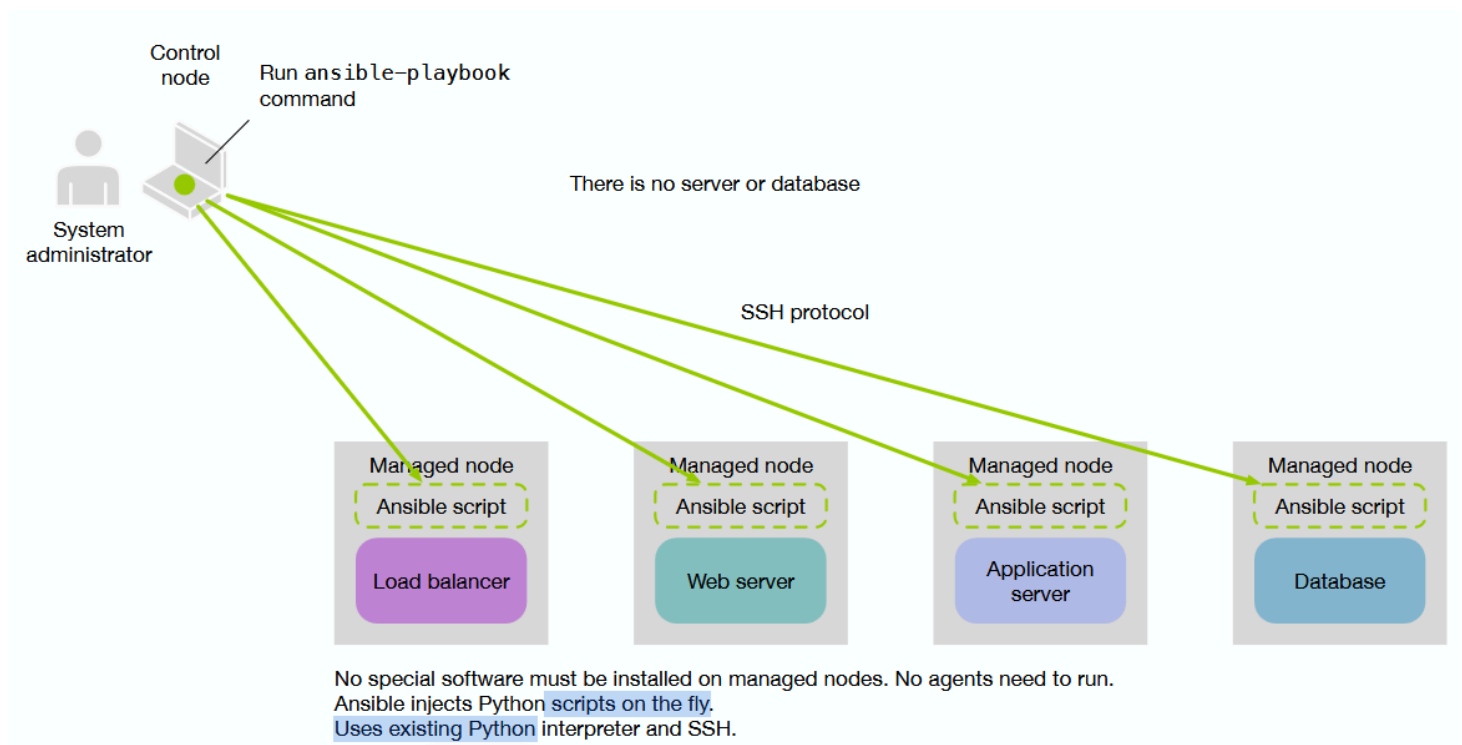
- prend le state comme input
- détermine le state actuel du système
- compare quoi faire
- se démerde pour faire l'action

Configuration Management Architectures

Puppet: client/server, agent-based



Ansible: agentless



Ansible

Automation engine for cloud provisioning, configuration management and application deployment
write in **PYTHON**

- collection of hosts = **inventory**
 - by default, describe it with a text file (inventory file)
- Host can be grouped, name = [...]
- playbook
 - play = host + task
- Roles = package for playbook, useful if we want to re-use things

Playbook

Play 1

```
- name: Base layer common to puppy and kitty node
hosts: all
tasks:
- name: Ensure NGINX is installed.
  apt:
    name: nginx
    state: present
    update_cache: yes
```

Play 2

```
- name: Configuration specific to puppy node
hosts: puppy
tasks:
- name: Ensure puppy group is present.
  group:
    name: puppy
    state: present
...
```

Play 3

```
- name: Configuration specific to kitty node
hosts: kitty
tasks:
- name: Ensure kitty group is present.
  group:
    name: kitty
    state: present
...
```

- `apt` - Installs or removes packages using the apt package manager.
- `yum` - Installs or removes packages using the yum package manager.
- `copy` - Copies a file from the control node to the managed node.
- `get_url` - Downloads a file from the web to the managed node.
- `template` - Generates a file from a template and copies it to the managed node.
- `lineinfile` - Ensures a particular line is in a file.
- `replace` - Replaces all instances of a particular string in a file using a back-referenced regular expression.
- `file` - Sets the attribute of a file, symlink, or directory.
- `service` - Starts, stops, or restarts a service.
- `user` - Manages OS user accounts.
- `command` - Executes command-line command on managed node. Avoid if possible.
- `debug` - Prints statements during execution for debugging.